

The
University of
Faisalabad

DEPARTMENT OF COMPUTER SCIENCES

DATA MINING

DS-303

LAB MANUAL

INSTRUCTOR

MR ARHAM

2023-BS-AI-020

MISHAL NADEEM

CONTENTS

Sr. No.	Lab No.	Topic	Page No.
1	Lab 1	Introduction to Data Mining	3
2	Lab 2	CRISP-DM Process	10
3	Lab 3	Handling Missing Values, Duplicates & Noise	17
4	Lab 4	Normalization, Standardization & Discretization	20
5	Lab 5	Market Basket Analysis	24
6	Lab 6	Apriori Algorithm	27
7	Lab 7	FP-Tree Construction & FP-Growth	30
8	Lab 8	Python Syntax & Basics	33
9	Lab 9	Decision Tree Classifier	38
10	Lab 10	Naïve Bayes & KNN	45
11	Lab 11	Support Vector Machine (SVM)	50
12	Lab 12	K-Means & Hierarchical Clustering	55
13	Lab 13	Outlier Detection	61
14	Lab 14	Data Scraping & Sentiment Analysis	67
15	Lab 15	Performance Comparison of Models	73

Lab 1

Introduction to Data Mining

Objective

To understand the fundamentals of Data Mining and become familiar with the KNIME analytics platform used for data processing, analysis, and machine learning.

1. Introduction to Data Mining

Data Mining is the process of extracting useful information, hidden patterns, and valuable knowledge from large datasets. It is an important field that combines techniques from statistics, machine learning, database systems, and artificial intelligence. Data mining helps organizations transform raw data into meaningful insights that support better decision-making.

Modern organizations generate enormous amounts of data through business transactions, social media, sensors, websites, and various digital systems. Data mining enables analysts to explore this data and discover relationships, trends, and patterns that would otherwise remain unnoticed.

2. Importance of Data Mining

- Converts raw data into useful information.
- Supports data-driven decision making.
- Identifies hidden patterns and trends.
- Improves business performance and efficiency.
- Helps predict future outcomes and behaviors.
- Assists in risk management and fraud detection.
- Enhances customer relationship management.

3. Data Mining Process

3.1 Data Collection

- Gathering data from multiple sources such as databases, spreadsheets, websites, and sensors.

3.2 Data Cleaning

- Removing missing values, duplicate records, and inconsistencies.
- Improving data quality for accurate analysis.

3.3 Data Transformation

- Converting data into a suitable format for analysis.
- Normalizing and organizing data structures.

3.4 Pattern Discovery

- Applying algorithms to identify relationships and trends within data.

3.5 Evaluation and Interpretation

- Analyzing results and validating discovered patterns.
- Converting findings into actionable knowledge.

4. Common Data Mining Techniques

Classification

- Assigns data into predefined categories or classes.
- Example: Email spam detection.

Clustering

- Groups similar data objects together.
- Example: Customer segmentation.

Association Rule Mining

- Discovers relationships between items in a dataset.
- Example: Market basket analysis.

Regression

- Predicts continuous numerical values.
- Example: House price prediction.

Anomaly Detection

- Identifies unusual or suspicious data patterns.
- Example: Credit card fraud detection.

6. Introduction to KNIME

KNIME (Konstanz Information Miner) is an open-source data analytics, reporting, and integration platform. It provides a graphical interface that allows users to create data analysis workflows using drag-and-drop components called nodes.

KNIME simplifies data mining and machine learning tasks by enabling users to build workflows visually without extensive programming knowledge.

7. Features of KNIME

Visual Workflow Design

- Drag-and-drop interface.
- Easy workflow creation and management.

Data Integration

- Supports CSV, Excel, databases, cloud storage, and web services.

Data Preprocessing

- Data cleaning and transformation tools.
- Missing value handling and filtering options.

Machine Learning Support

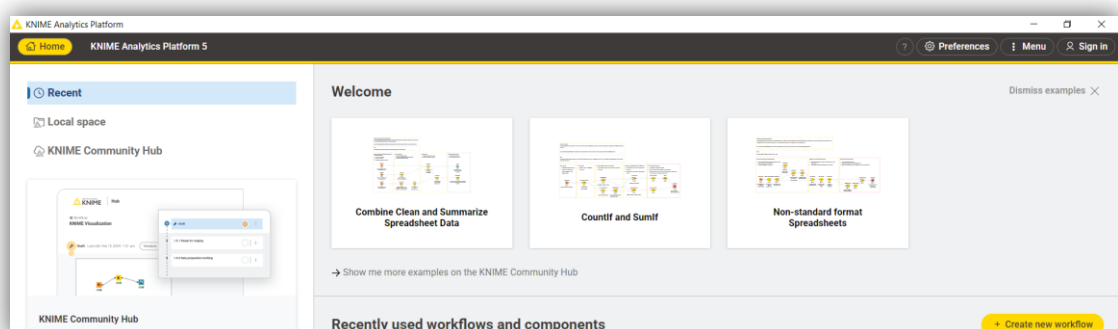
- Classification algorithms.
- Clustering algorithms.
- Regression models.
- Model evaluation tools.

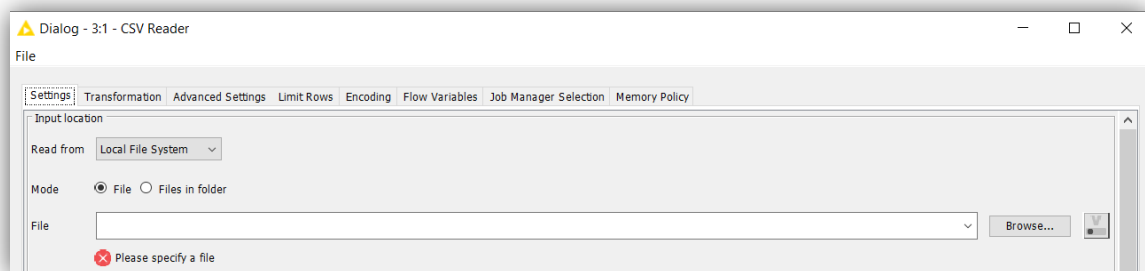
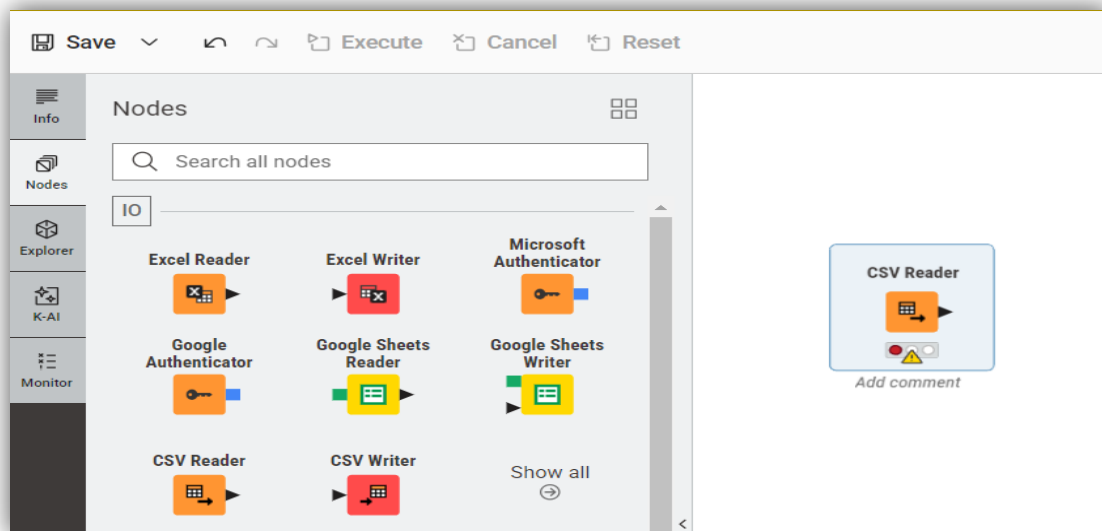
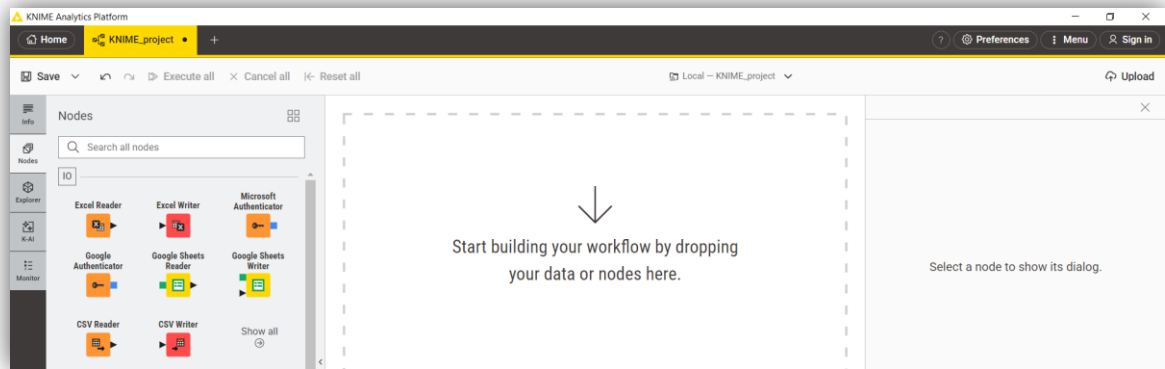
Data Visualization

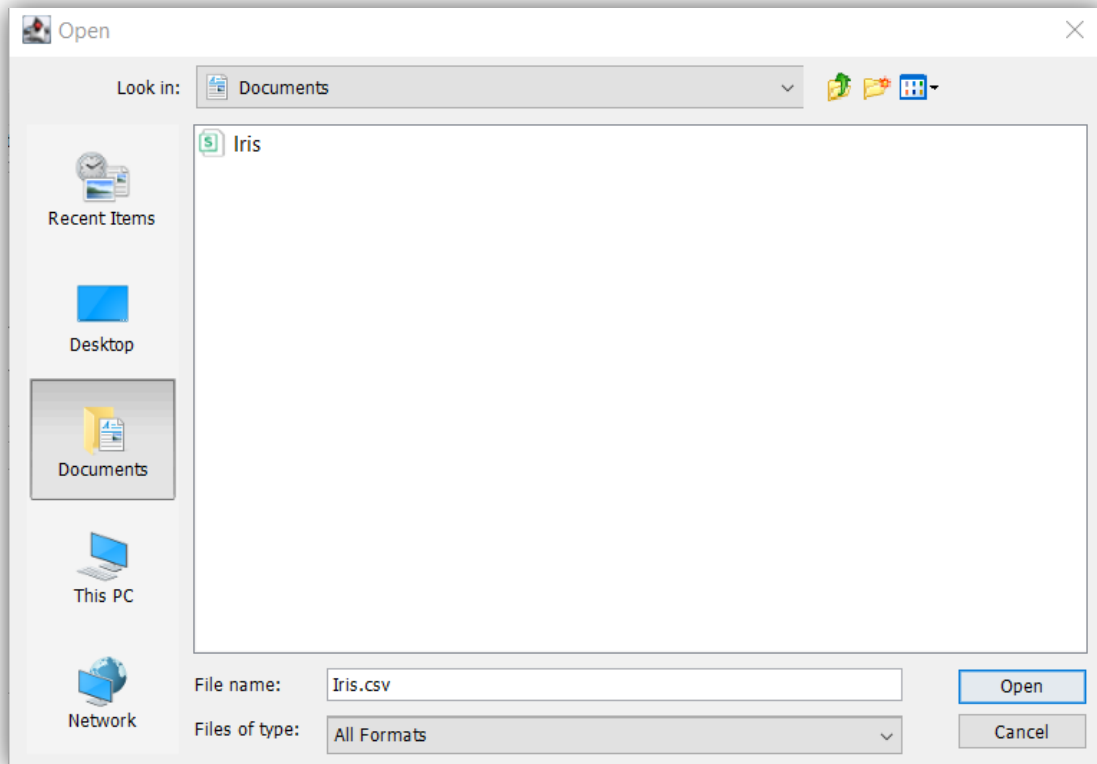
- Charts and graphs for data exploration.
- Interactive dashboards and reports.

Integration with Programming Languages

- Supports Python integration.
- Supports R integration.
- Extensible through community plugins.







► 1: File Table Flow Variables

Rows: 156 | Columns: 6 Table Statistics

#	RowID	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
		Number (Float)	Number (Float)	Number (Float)	Number (Float)	Number (Float)	String
1	Row0	1	5.1	3.5	1.4	0.2	Iris-setosa
2	Row1	2	4.9	3	1.4	0.2	Iris-setosa
3	Row2	3	4.7	3.2	1.3	0.2	Iris-setosa
4	Row3	4	4.6	3.1	1.5	0.2	Iris-setosa
5	Row4	5	5	3.6	1.4	0.2	Iris-setosa
6	Row5	6	5.4	3.9	1.7	0.4	Iris-setosa

Nodes

Info

Nodes

Views Analytics +21

Statistics

Statistics View

Tree Ensemble Statistics

Matrix Distance

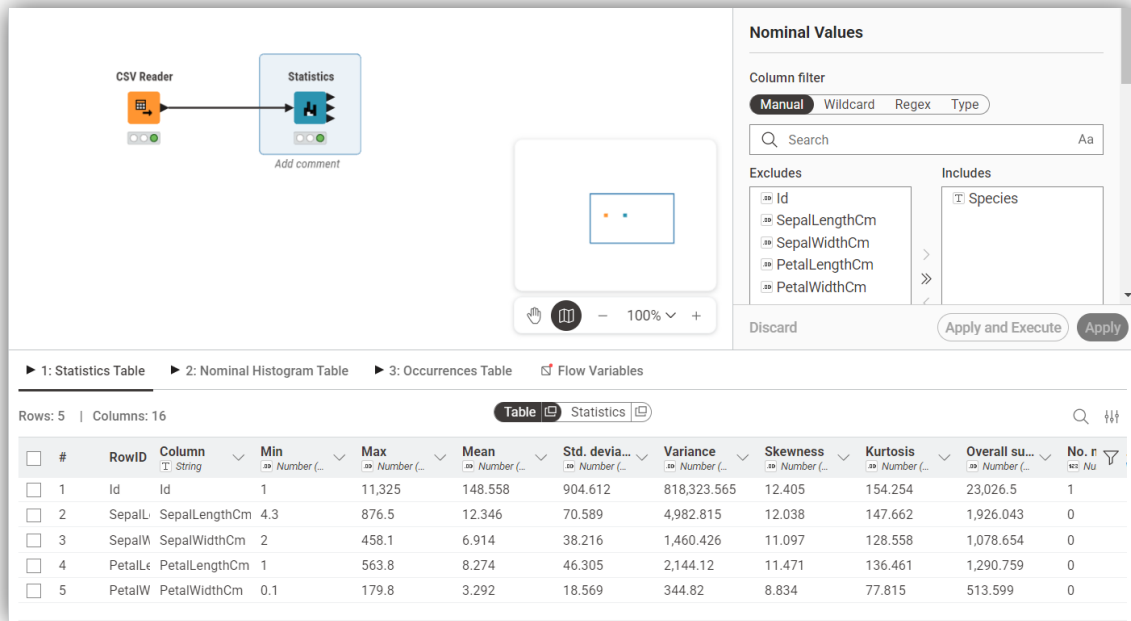
Numeric Distances

Ungroup

CSV Reader

Statistics

Add comment



Nominal Values

Column filter: Manual Wildcard Regex Type

Search: Aa

Excludes: Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm

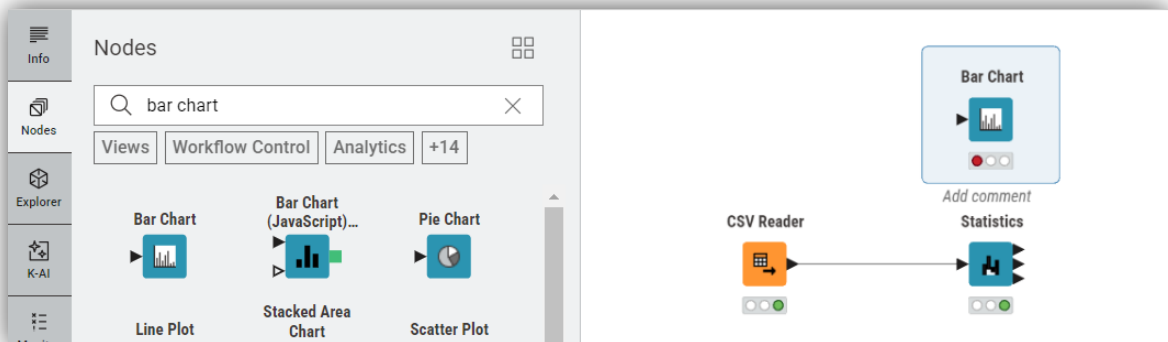
Includes: Species

Discard Apply and Execute Apply

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table Flow Variables

Rows: 5 | Columns: 16

#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis	Overall su...	No. r
1	Id	Id	1	11,325	148.558	904.612	818,323.565	12.405	154.254	23,026.5	1
2	SepalL	SepalLengthCm	4.3	876.5	12.346	70.589	4,982.815	12.038	147.662	1,926.043	0
3	SepalW	SepalWidthCm	2	458.1	6.914	38.216	1,460.426	11.097	128.558	1,078.654	0
4	PetalL	PetalLengthCm	1	563.8	8.274	46.305	2,144.12	11.471	136.461	1,290.759	0
5	PetalW	PetalWidthCm	0.1	179.8	3.292	18.569	344.82	8.834	77.815	513.599	0



Nodes

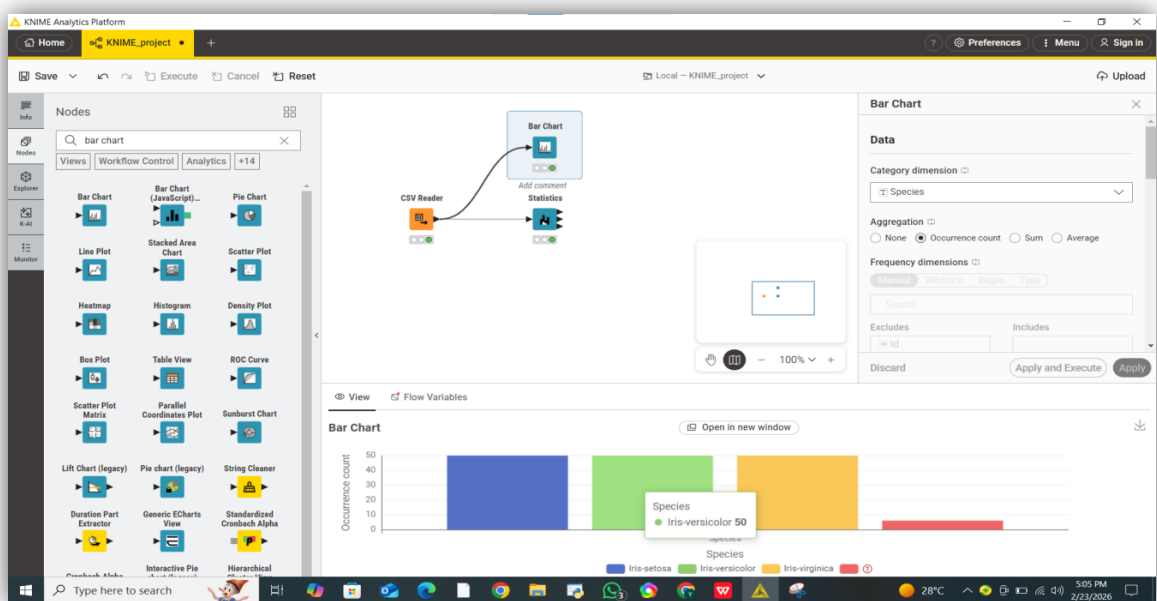
Search: bar chart

Views Workflow Control Analytics +14

Bar Chart Bar Chart (JavaScript)... Pie Chart

Line Plot Stacked Area Chart Scatter Plot

CSV Reader Statistics Bar Chart



KNIME Analytics Platform

Save Execute Cancel Reset

Local - KNIME_project

Nodes

Search: bar chart

Views Workflow Control Analytics +14

Bar Chart Bar Chart (JavaScript)... Pie Chart

Line Plot Stacked Area Chart Scatter Plot

Heatmap Histogram Density Plot

Box Plot Table View ROC Curve

Scatter Plot Matrix Parallel Coordinates Plot Sunburst Chart

Lift Chart (legacy) Pie Chart (legacy) String Cleaner

Duration Part Extractor Generic ECharts View Standardized Crossback Alpha

Interactive Pie Hierarchical

Bar Chart

Data

Category dimension: Species

Aggregation: None Occurrence count Sum Average

Frequency dimensions: Manual Wildcard Regex Type

Search: Aa

Excludes: Id Includes: Species

Discard Apply and Execute Apply

Bar Chart

Occurrence count

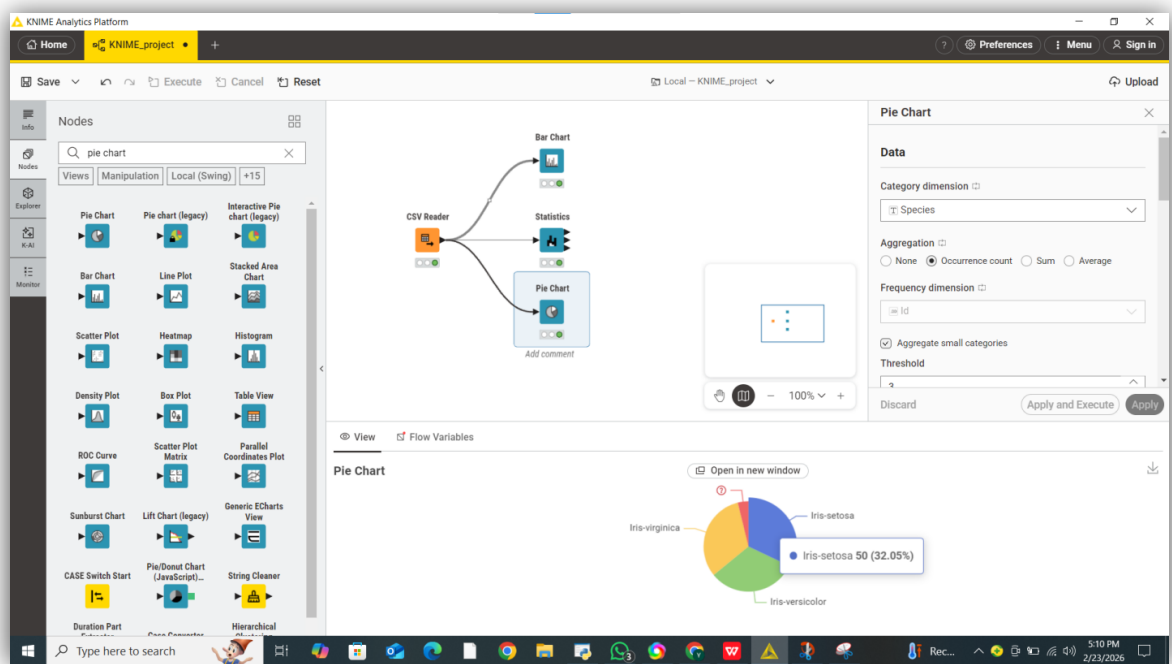
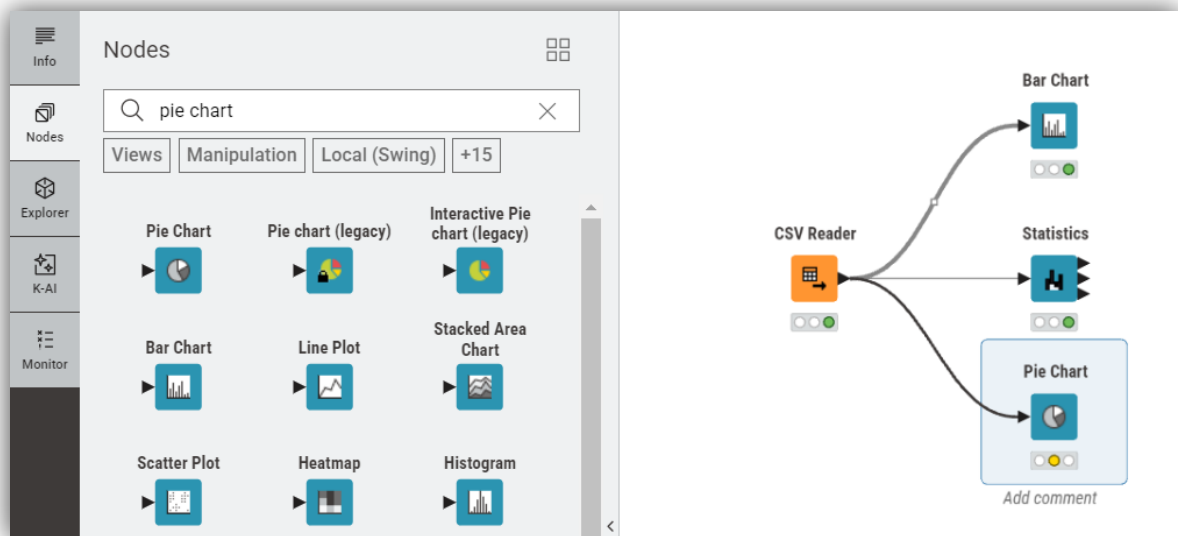
Species

Iris-setosa Iris-versicolor Iris-virginica

Species: Iris-versicolor 50

Open in new window

28°C 5:05 PM 2/23/2026



Lab 2

CRISP-DM Process

Objective

To understand the CRISP-DM methodology and explore the basic features of Orange Data Mining software.

1. Introduction to CRISP-DM

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a widely used framework for managing data mining projects. It provides a structured approach for transforming business problems into data-driven solutions. The methodology is flexible and can be applied across different industries and domains.

2. Phases of CRISP-DM

2.1 Business Understanding

- Define business objectives and project goals.
- Identify project requirements and success criteria.

2.2 Data Understanding

- Collect and explore data.
- Identify data quality issues and patterns.

2.3 Data Preparation

- Clean and transform data.
- Handle missing values and select relevant features.

2.4 Modeling

- Apply machine learning or data mining algorithms.
- Train and test models.

2.5 Evaluation

- Assess model performance.
- Verify that results meet business objectives.

2.6 Deployment

- Implement the model for practical use.

- Monitor and maintain system performance.

4. Introduction to Orange Software

Orange is an open-source data mining and machine learning tool that uses a visual programming approach. Users can create workflows by connecting widgets through a drag-and-drop interface, making data analysis easier without extensive coding.

5. Features of Orange

Visual Workflow Design

- Drag-and-drop interface.
- Easy workflow creation.

Data Preprocessing

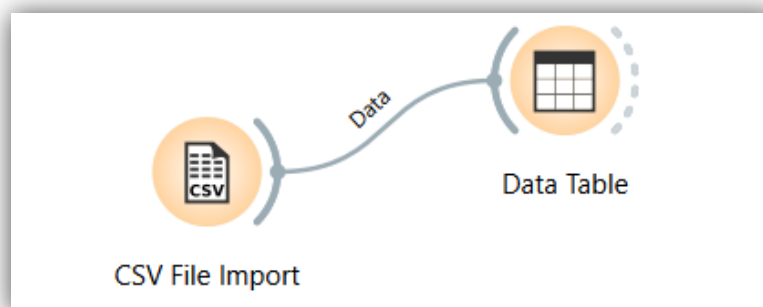
- Data cleaning and transformation.
- Feature selection tools.

Machine Learning

- Classification, clustering, and regression algorithms.
- Model evaluation capabilities.

Data Visualization

- Interactive charts and graphs.
- Easy exploration of datasets.



Data Table - Orange

Info
1000 instances (no missing data)
16 features
No target variable.
1 meta attribute

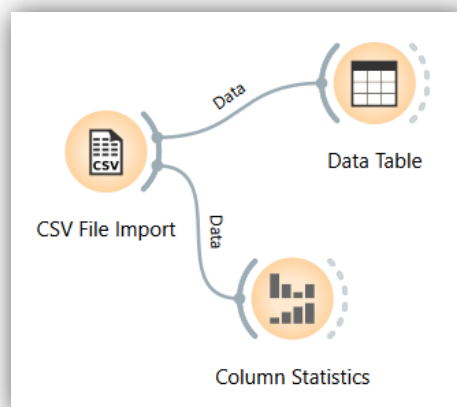
Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

	Invoice ID	Branch	City	Customer type	Gender
1	750-67-8428	Alex	Yangon	Member	Female
2	226-31-3081	Giza	Naypyitaw	Normal	Female
3	631-41-3108	Alex	Yangon	Normal	Female
4	123-19-1176	Alex	Yangon	Member	Female
5	373-73-7910	Alex	Yangon	Member	Female
6	699-14-3026	Giza	Naypyitaw	Member	Female
7	355-53-5943	Alex	Yangon	Member	Female
8	315-22-5665	Giza	Naypyitaw	Member	Female
9	665-32-9167	Alex	Yangon	Member	Female
10	692-92-5582	Cairo	Mandalay	Member	Female
11	351-62-0822	Cairo	Mandalay	Member	Female
12	529-56-3974	Cairo	Mandalay	Member	Female
13	365-64-0515	Alex	Yangon	Member	Female
14	252-56-2699	Alex	Yangon	Member	Female
15	829-34-3910	Alex	Yangon	Member	Female
16	299-46-1805	Cairo	Mandalay	Member	Female
17	656-95-9349	Alex	Yangon	Member	Female
18	765-26-6051	Alex	Yangon	Member	Female

1000 | 1000 | 1000

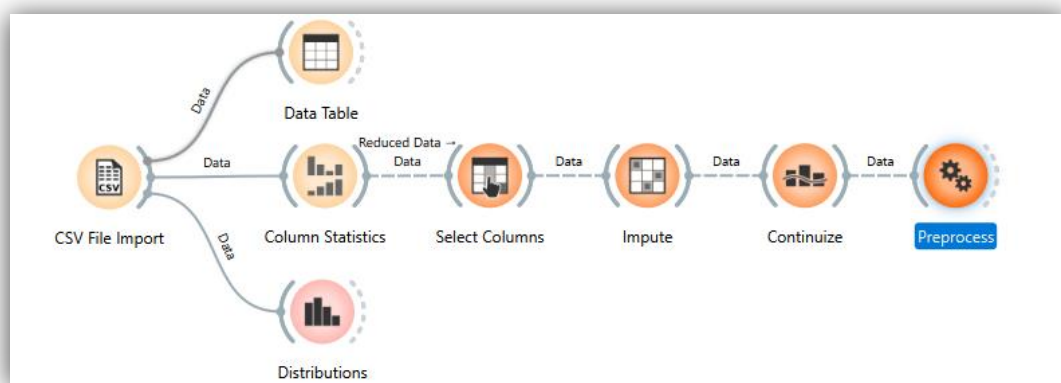
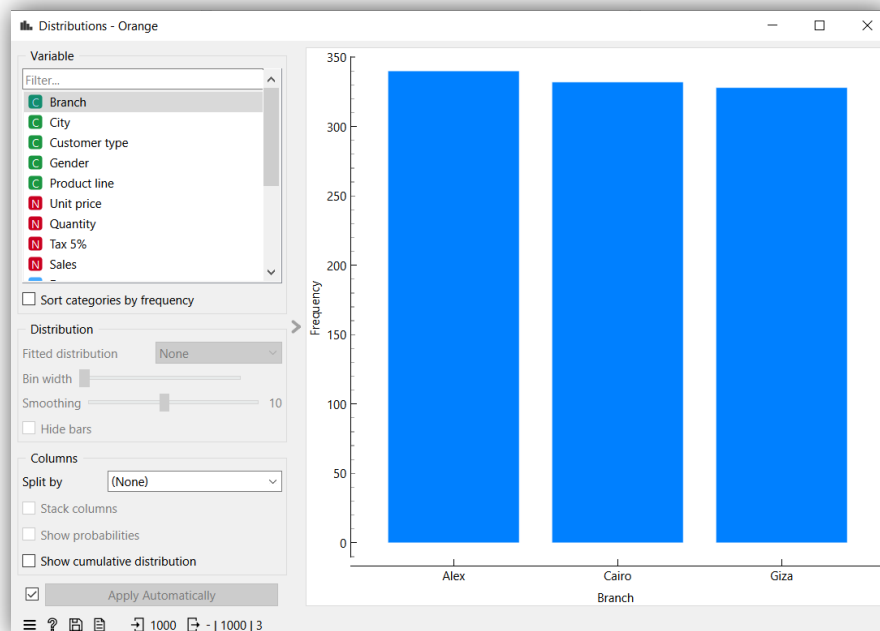
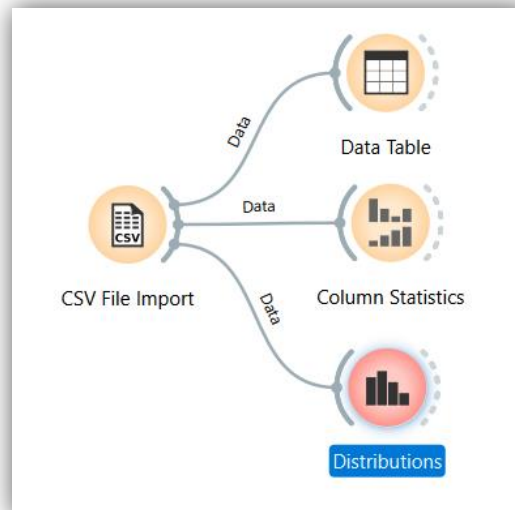


Column Statistics - Orange

Column	Distribution	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
Unit price		55.6721	83.77	55.23	0.475667	10.08	99.96	0 (0 %)
Quantity		5.51	10	5	0.53	1	10	0 (0 %)
Tax 5%		15.3794	4.154	12.088	0.760953	0.5085	49.65	0 (0 %)
Sales		322.967	87.234	253.848	0.760953	10.6785	1042.65	0 (0 %)
cogs		307.587	83.08	241.76	0.760953	10.17	993	0 (0 %)
gross margin percentage		4.7619	4.7619	4.7619	1.28697e-14	4.7619	4.7619	0 (0 %)

Color: None ☒ Send Automatically

1000 | 1000 | 1000





Test and Score - Orange

☒ Cross validation

Number of folds: 5

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	0.484	0.147	0.146	0.147	0.147	-0.000

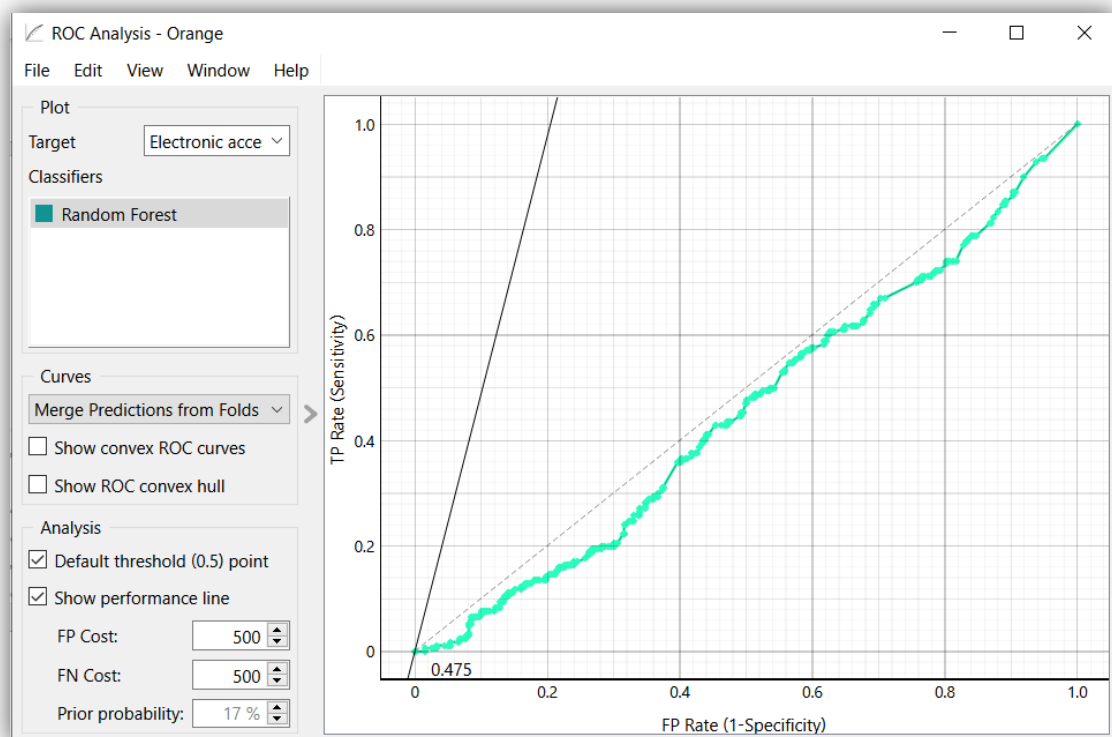
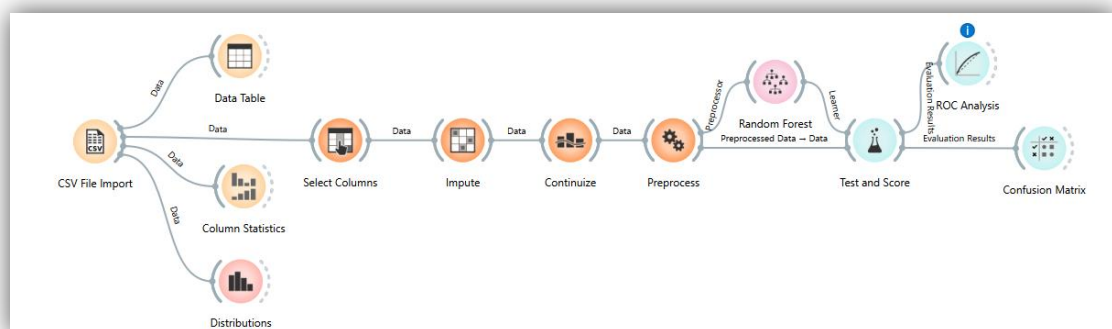
Confusion Matrix - Orange

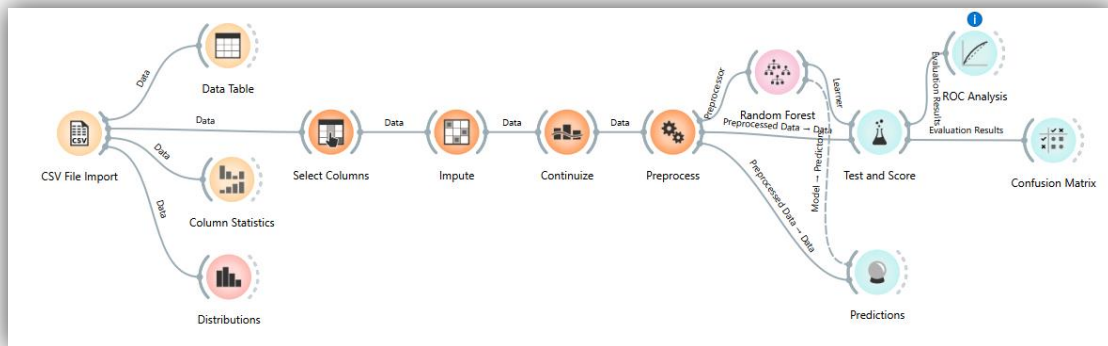
File View Window Help

Learners

Random Forest

		Predicted						Σ
		Electronic accessories	Fashion accessories	Food and beverages	Health and beauty	Home and lifestyle	Sports and travel	
Actual	Electronic accessories	24	28	30	23	34	31	170
	Fashion accessories	41	30	30	19	27	31	178
	Food and beverages	32	41	23	21	25	32	174
	Health and beauty	27	27	29	17	23	29	152
	Home and lifestyle	23	34	29	21	24	29	160
	Sports and travel	40	26	27	15	29	29	166
Σ		187	186	168	116	162	181	1000





Predictions - Orange

File View Window Help

Show probabilities for Classes in data ☒ Show classification errors Restore Original Order

	Product line	Invoice ID	Branch=Cairo	Branch=Giza	City=Naypyitaw	Class
	Health and beauty	750-67-8428	0	0	0	1
	Electronic accessories	226-31-3081	0	1	1	0
	Home and lifestyle	631-41-3108	0	0	0	1
	Health and beauty	123-19-1176	0	0	0	1
	Sports and travel	373-73-7910	0	0	0	1
	Electronic accessories	699-14-3026	0	1	1	0
	Electronic accessories	355-53-5943	0	0	0	1
	Home and lifestyle	315-22-5665	0	1	1	0
	Health and beauty	665-32-9167	0	0	0	1

☒ Show performance scores Target class: (Average over classes)

1000 | - 1000 | 1000 | -

Lab 3

Handling Missing Values, Duplicates & Noise

Objective

To understand the importance of data cleaning and learn techniques for handling missing values, duplicate records, and noisy data in a dataset.

1. Introduction

Data preprocessing is an essential step in data mining and machine learning. Real-world datasets often contain missing values, duplicate records, and noisy data that can negatively affect the accuracy and reliability of analysis. Data cleaning helps improve data quality and ensures better results from data mining models.

2. Handling Missing Values

Missing values occur when information for one or more attributes is unavailable.

Causes of Missing Values

- Data entry errors.
- Equipment or sensor failures.
- Incomplete surveys or forms.
- Data collection issues.

Methods for Handling Missing Values

- Removing records with missing values.
- Replacing missing values with mean, median, or mode.
- Using prediction techniques to estimate missing values.
- Filling values based on domain knowledge.

3. Handling Duplicate Records

Duplicate records occur when the same data entry appears multiple times in a dataset.

Causes of Duplicates

- Repeated data entry.
- Data integration from multiple sources.
- System or processing errors.

Methods for Handling Duplicates

- Identifying duplicate rows.
- Removing repeated records.
- Using unique identifiers for verification.

4. Handling Noisy Data

Noisy data refers to incorrect, inconsistent, or irrelevant values present in a dataset.

Causes of Noise

- Human errors.
- Faulty data collection devices.
- Transmission errors.
- Measurement inaccuracies.

Load dataset

```
[2]: df = pd.read_csv("dataset 4.csv")
```

Check missing values

```
[3]: df.isnull().sum()
```

```
[3]: Loan_ID      0
      Gender      11
      Married     0
      Dependents  10
      Education   0
      Self_Employed 23
      ApplicantIncome 0
      CoapplicantIncome 0
      LoanAmount   5
      Loan_Amount_Term 6
      Credit_History 29
      Property_Area 0
      dtype: int64
```

Fill missing numerical values with mean

```
[4]: df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
[4]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Proq
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.000000	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.000000	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.000000	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	0.825444	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.000000	
...	...	...	...	...	...	...	...	...	...	...	...	...
362	LP002971	Male	Yes	3+	Not Graduate	Yes	4009	1777	113.0	360.0	1.000000	
363	LP002975	Male	Yes	0	Graduate	No	4158	709	115.0	360.0	1.000000	
364	LP002980	Male	No	0	Graduate	No	3250	1993	126.0	360.0	0.825444	
365	LP002986	Male	Yes	0	Graduate	No	5000	2393	158.0	360.0	1.000000	
366	LP002989	Male	No	0	Graduate	Yes	9200	0	98.0	180.0	1.000000	

367 rows x 12 columns

Display cleaned data

```
[5]: df.head()
```

```
[5]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.000000	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.000000	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.000000	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	0.825444	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.000000	

Check duplicate rows

```
[6]: "Duplicate rows:", df.duplicated().sum()
```

```
[6]: ('Duplicate rows:', np.int64(0))
```

Remove duplicates

```
[7]: df = df.drop_duplicates()
```

Display updated dataset

```
[8]: df.head()
```

```
[8]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.000000	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.000000	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.000000	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	0.825444	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.000000	

Remove outliers from numeric columns

```
[9]: for column in df.select_dtypes(include=['int64', 'float64']).columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[column] >= lower) & (df[column] <= upper)]
```

Display cleaned dataset

```
[10]: df.head()
```

```
[10]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	
5	LP001054	Male	Yes	0	Not Graduate	Yes	2165	3422	152.0	360.0	1.0	

Lab 4

Normalization, Standardization & Discretization

Objective

To understand the techniques of Normalization, Standardization, and Discretization used in data preprocessing for improving data quality and model performance.

1. Introduction

Data preprocessing is an important step in data mining and machine learning. Datasets often contain attributes with different scales and formats, which can affect the performance of analytical models. Normalization, Standardization, and Discretization are preprocessing techniques used to transform data into a suitable form for analysis.

2. Normalization

Normalization is the process of scaling numerical values to a specific range, usually between 0 and 1. It ensures that all attributes contribute equally to the analysis regardless of their original scale.

Purpose

- Bring data to a common scale.
- Reduce the impact of large numerical values.
- Improve algorithm performance.

3. Standardization

Standardization is the process of transforming data so that it has a mean of 0 and a standard deviation of 1. It is commonly used when data follows a normal distribution.

Purpose

- Remove scale differences between attributes.
- Center data around the mean.
- Improve model accuracy.

4. Discretization

Discretization is the process of converting continuous numerical values into a limited number of categories or intervals. It simplifies data and makes patterns easier to understand.

Purpose

- Transform continuous data into categorical data.
- Reduce data complexity.
- Improve interpretability.

Python Basics

```
[1]: import pandas as pd

class DataProcessor:
    def __init__(self, file_path):
        self.file_path = file_path
        self.data = None

    def load_data(self):
        self.data = pd.read_csv(self.file_path)
        return self.data

    def show_info(self):
        info = {
            "Rows": self.data.shape[0],
            "Columns": self.data.shape[1],
            "Column Names": list(self.data.columns)
        }
        return info

    def handle_missing_values(self):
        self.data = self.data.fillna(self.data.mean(numeric_only=True))
        return self.data

    def remove_duplicates(self):
        self.data = self.data.drop_duplicates()
        return self.data

    def normalize_column(self, column_name):
        self.data[column_name] = (
            self.data[column_name] - self.data[column_name].min()
        ) / (
            self.data[column_name].max() - self.data[column_name].min()
        )
        return self.data

    def get_summary(self):
        return self.data.describe()
```

```

Lists

[2]: selected_columns = ['Age', 'Income', 'LoanAmount']

Dictionary

[3]: column_info = {
      "numeric": ['Age', 'Income', 'LoanAmount'],
      "categorical": ['Gender', 'Education']
    }

print("Selected Columns:", selected_columns)
print("Column Info:", column_info)

Selected Columns: ['Age', 'Income', 'LoanAmount']
Column Info: {'numeric': ['Age', 'Income', 'LoanAmount'], 'categorical': ['Gender', 'Education']}

Load dataset

[4]: processor = DataProcessor("dataset 1.csv")
df = processor.load_data()

Display dataset info

[5]: print("\nDataset Info:")
processor.show_info()

Dataset Info:
[5]: {'Rows': 1888,
      'Columns': 17,
      'Column Names': ['Invoice ID',
                        'Branch',
                        'City',
                        'Customer type',
                        'Gender',
                        'Product line',
                        'Unit price',
                        'Quantity',
                        'Tax 5%',
                        'Sales',
                        'Date',
                        'Time',
                        'Payment',
                        'cogs',
                        'gross margin percentage',
                        'gross income',
                        'Rating']}

```

Preprocessing

```
[6]: processor.handle_missing_values()
processor.remove_duplicates()
```

```
[6]:
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	4
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29:00 AM	Cash	76.40	4.761905	3.8200	4
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	1:23:00 PM	Credit card	324.31	4.761905	16.2155	4
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	4
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	Giza	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675	1/29/2019	1:46:00 PM	Ewallet	40.35	4.761905	2.0175	4
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900	3/2/2019	5:16:00 PM	Ewallet	973.80	4.761905	48.6900	4
997	727-02-1313	Alex	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	33.4320	2/9/2019	1:22:00 PM	Cash	31.84	4.761905	1.5920	4
998	347-56-2442	Alex	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110	2/22/2019	3:33:00 PM	Cash	65.82	4.761905	3.2910	4
999	849-09-3807	Alex	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990	2/18/2019	1:28:00 PM	Cash	618.38	4.761905	30.9190	4

1000 rows x 17 columns

Normalize one column

```
[7]: processor.normalize_column("Rating")
```

```
[7]:
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	0.850000
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29:00 AM	Cash	76.40	4.761905	3.8200	0.933333
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	1:23:00 PM	Credit card	324.31	4.761905	16.2155	0.566667
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	0.733333
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	0.216667
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	233-67-5758	Giza	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.0175	42.3675	1/29/2019	1:46:00 PM	Ewallet	40.35	4.761905	2.0175	0.366667
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.6900	1022.4900	3/2/2019	5:16:00 PM	Ewallet	973.80	4.761905	48.6900	0.066667
997	727-02-1313	Alex	Yangon	Member	Male	Food and beverages	31.84	1	1.5920	33.4320	2/9/2019	1:22:00 PM	Cash	31.84	4.761905	1.5920	0.616667
998	347-56-2442	Alex	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.2910	69.1110	2/22/2019	3:33:00 PM	Cash	65.82	4.761905	3.2910	0.016667
999	849-09-3807	Alex	Yangon	Member	Female	Fashion accessories	88.34	7	30.9190	649.2990	2/18/2019	1:28:00 PM	Cash	618.38	4.761905	30.9190	0.433333

1000 rows x 17 columns

Display processed data

```
[8]: print("\nProcessed Data:")
processor.data.head()
```

Processed Data:

```
[8]:
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Sales	Date	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	1:08:00 PM	Ewallet	522.83	4.761905	26.1415	0.850000
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29:00 AM	Cash	76.40	4.761905	3.8200	0.933333
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	1:23:00 PM	Credit card	324.31	4.761905	16.2155	0.566667
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	8:33:00 PM	Ewallet	465.76	4.761905	23.2880	0.733333
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	10:37:00 AM	Ewallet	604.17	4.761905	30.2085	0.216667

Summary statistics

```
[9]: print("\nSummary Statistics:")
processor.get_summary()
```

Summary Statistics:

```
[9]:
```

	Unit price	Quantity	Tax 5%	Sales	cogs	gross margin percentage	gross income	Rating
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1.000000e+03	1000.000000	1000.000000
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905e+00	15.379369	0.49545
std	26.494628	2.923431	11.708825	245.885335	234.17651	6.131498e-14	11.708825	0.28643
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905e+00	0.508500	0.000000
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905e+00	5.924875	0.25000
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905e+00	12.088000	0.50000
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905e+00	22.445250	0.75000
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905e+00	49.650000	1.00000

Lab 5

Market Basket Analysis

Objective

To understand the concept of Market Basket Analysis and its role in discovering relationships between items purchased together in transactional data.

1. Introduction

Market Basket Analysis (MBA) is a data mining technique used to identify associations and relationships between products frequently purchased together by customers. It helps businesses understand customer buying behavior and discover patterns within transaction data.

The technique is widely used in retail stores, supermarkets, and e-commerce platforms to improve sales, marketing strategies, and product placement.

2. Working of Market Basket Analysis

Market Basket Analysis examines customer transactions to find combinations of items that occur together frequently. Based on these patterns, businesses can make recommendations and promotional offers.

Process

- Collect transaction data.
- Identify frequent itemsets.
- Generate association rules.
- Analyze relationships between products.
- Apply findings to business decisions.

3. Key Measures

1. Support
2. Confidence
3. Lift

4. Association Rules

Association rules are expressed in the form:

If Item A is purchased, then Item B is also likely to be purchased.

Example

- Bread → Butter
- Laptop → Mouse
- Tea → Biscuits

Step 2: Market Basket Analysis

Import libraries

```
[1]: import pandas as pd
     from mlxtend.frequent_patterns import apriori, association_rules
```

Load dataset

```
[2]: df = pd.read_csv("dataset 5.csv")
```

View data

```
[3]: df.head()
```

```
[3]:
```

	Invoice	Item
0	1	Bread
1	1	Milk
2	1	Eggs
3	2	Bread
4	2	Butter

Step 3: Prepare Data

Convert data into transaction format

```
[4]: basket = df.groupby(['Invoice', 'Item'])['Item'] \
     .count().unstack().reset_index().fillna(0)
```

Convert values to 0 and 1

```
[5]: basket = df.groupby(['Invoice', 'Item']).size().unstack(fill_value=0)
     basket = (basket > 0).astype(int)
```

```
[6]: basket.head()
```

```
[6]:
```

	Item	Bread	Butter	Eggs	Milk
Invoice					
1	1	1	0	1	1
2	1	1	1	0	0
3	0	1	1	1	1
4	1	0	0	0	1
5	0	1	1	1	0

Step 4: Apply Apriori Algorithm

```
[7]: frequent_itemsets = apriori(basket, min_support=0.02, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.7	frozenset({Bread})
1	0.6	frozenset({Butter})
2	0.5	frozenset({Eggs})
3	0.6	frozenset({Milk})
4	0.3	frozenset({Bread, Butter})
5	0.3	frozenset({Bread, Eggs})
6	0.4	frozenset({Bread, Milk})
7	0.2	frozenset({Eggs, Butter})
8	0.3	frozenset({Milk, Butter})
9	0.3	frozenset({Eggs, Milk})
10	0.1	frozenset({Bread, Milk, Butter})
11	0.2	frozenset({Bread, Eggs, Milk})
12	0.1	frozenset({Eggs, Milk, Butter})

Step 5: Generate Association Rules

```
[8]: rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
```

```
[9]: print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

	antecedents	consequents	support	confidence	\
0	frozenset({Eggs})	frozenset({Milk})	0.3	0.600000	
1	frozenset({Milk})	frozenset({Eggs})	0.3	0.500000	
2	frozenset({Bread, Eggs})	frozenset({Milk})	0.2	0.666667	
3	frozenset({Bread, Milk})	frozenset({Eggs})	0.2	0.500000	
4	frozenset({Eggs})	frozenset({Bread, Milk})	0.2	0.400000	
5	frozenset({Milk})	frozenset({Bread, Eggs})	0.2	0.333333	

	lift
0	1.000000
1	1.000000
2	1.111111
3	1.000000
4	1.000000
5	1.111111

Lab 6

Apriori Algorithm

Objective

To understand the Apriori Algorithm and its use in discovering frequent itemsets and association rules from transactional data.

1. Introduction

The Apriori Algorithm is one of the most widely used algorithms in data mining for association rule mining and Market Basket Analysis. It is designed to identify frequent itemsets in a dataset and generate association rules that reveal relationships among items.

The algorithm is based on the principle that if an itemset is frequent, then all of its subsets must also be frequent. This property helps reduce the search space and improves efficiency.

2. Working of Apriori Algorithm

Apriori works by scanning the transaction database multiple times to find item combinations that satisfy a minimum support threshold.

Steps of the Algorithm

1. Generate frequent 1-itemsets.
2. Remove itemsets that do not meet minimum support.
3. Generate candidate 2-itemsets from frequent 1-itemsets.
4. Repeat the process for larger itemsets.
5. Generate association rules from the frequent itemsets.

3. Key Concepts

Frequent Itemset

A group of items that appears together frequently in transactions.

Support

- Measures how often an itemset appears in the dataset.
- Used to identify frequent itemsets.

Confidence

- Measures the reliability of an association rule.
- Indicates how often the rule is correct.

Lift

- Measures the strength of the relationship between items.
- Helps determine whether associations are meaningful.

4. Apriori Principle

The Apriori Principle states:

"If an itemset is frequent, then all of its subsets must also be frequent."

This principle allows the algorithm to eliminate itemsets that cannot possibly be frequent, reducing computational effort.

Import Libraries

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules
```

Create Dataset

```
[2]: data = [
      ['Milk', 'Bread', 'Butter'],
      ['Bread', 'Butter'],
      ['Milk', 'Bread'],
      ['Milk', 'Butter'],
      ['Bread', 'Butter'],
      ['Milk', 'Bread', 'Butter'],
      ['Milk', 'Bread'],
      ['Bread'],
      ['Milk', 'Butter'],
      ['Milk', 'Bread', 'Butter']
      ]
```

Convert to One-Hot Encoding

```
[3]: te = TransactionEncoder()
      te_data = te.fit(data).transform(data)

      df = pd.DataFrame(te_data, columns=te.columns_)
      df
```

```
[3]:
```

	Bread	Butter	Milk
0	True	True	True
1	True	True	False
2	True	False	True
3	False	True	True
4	True	True	False
5	True	True	True
6	True	False	True
7	True	False	False
8	False	True	True
9	True	True	True

Apply Apriori Algorithm

```
[4]: frequent_items = apriori(df, min_support=0.3, use_colnames=True)
     frequent_items
```

```
[4]:
```

	support	itemsets
0	0.8	frozenset([Bread])
1	0.7	frozenset([Butter])
2	0.7	frozenset([Milk])
3	0.5	frozenset([Butter, Bread])
4	0.5	frozenset([Milk, Bread])
5	0.5	frozenset([Butter, Milk])
6	0.3	frozenset([Butter, Milk, Bread])

Generate Association Rules

```
[5]: rules = association_rules(frequent_items, metric="confidence", min_threshold=0.6)
     rules
```

```
[5]:
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certain
0	frozenset([Butter])	frozenset([Bread])	0.7	0.8	0.5	0.714286	0.892857	1.0	-0.06	0.70	-0.285714	0.500000	-0.4285
1	frozenset([Bread])	frozenset([Butter])	0.8	0.7	0.5	0.625000	0.892857	1.0	-0.06	0.80	-0.375000	0.500000	-0.2500
2	frozenset([Bread])	frozenset([Milk])	0.8	0.7	0.5	0.625000	0.892857	1.0	-0.06	0.80	-0.375000	0.500000	-0.2500
3	frozenset([Milk])	frozenset([Bread])	0.7	0.8	0.5	0.714286	0.892857	1.0	-0.06	0.70	-0.285714	0.500000	-0.4285
4	frozenset([Butter])	frozenset([Milk])	0.7	0.7	0.5	0.714286	1.020408	1.0	0.01	1.05	0.066667	0.555556	0.0476
5	frozenset([Milk])	frozenset([Butter])	0.7	0.7	0.5	0.714286	1.020408	1.0	0.01	1.05	0.066667	0.555556	0.0476
6	frozenset([Butter, Bread])	frozenset([Milk])	0.5	0.7	0.3	0.600000	0.857143	1.0	-0.05	0.75	-0.250000	0.333333	-0.3333
7	frozenset([Butter, Milk])	frozenset([Bread])	0.5	0.8	0.3	0.600000	0.750000	1.0	-0.10	0.50	-0.400000	0.300000	-1.0000
8	frozenset([Milk, Bread])	frozenset([Butter])	0.5	0.7	0.3	0.600000	0.857143	1.0	-0.05	0.75	-0.250000	0.333333	-0.3333

Lab 7

FP-Tree Construction & FP-Growth

Objective

To understand the construction of an FP-Tree (Frequent Pattern Tree) and the working of the FP-Growth algorithm for discovering frequent itemsets efficiently.

1. Introduction

FP-Growth (Frequent Pattern Growth) is a popular data mining algorithm used for frequent itemset mining and association rule learning. It was developed as an alternative to the Apriori Algorithm to overcome the problem of generating a large number of candidate itemsets.

The FP-Growth algorithm uses a compact data structure called the **FP-Tree (Frequent Pattern Tree)** to store transaction data and discover frequent patterns efficiently.

2. FP-Tree (Frequent Pattern Tree)

An FP-Tree is a tree-based structure that compresses transaction data while preserving itemset association information.

Characteristics of FP-Tree

- Compact representation of transaction data.
- Stores only frequent items.
- Reduces database scans.
- Preserves item relationships.
- Improves mining efficiency.

3. Construction of FP-Tree

The FP-Tree is constructed through the following steps:

Step 1: Scan the Database

- Count the frequency of each item.
- Remove items that do not satisfy the minimum support threshold.

Step 2: Sort Frequent Items

- Arrange items in descending order of frequency.

Step 3: Create the Tree

- Insert transactions into the tree.
- Share common prefixes among transactions.
- Update node counts accordingly.

Step 4: Create Header Table

- Maintain links to similar items in different branches of the tree.

Import Libraries

```
[1]: from collections import defaultdict
```

Define FP-Tree Node Class

```
[2]: class FPNode:
    def __init__(self, item, count, parent):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None # Link to next node

    def increment(self, count):
        self.count += count
```

Build FP-Tree Function

```
[3]: def build_fp_tree(transactions, min_support):
    item_count = defaultdict(int)

    # Count frequency
    for transaction in transactions:
        for item in transaction:
            item_count[item] += 1

    # Remove infrequent items
    item_count = {k: v for k, v in item_count.items() if v >= min_support}
    if len(item_count) == 0:
        return None, None

    # Header table
    header_table = {k: [v, None] for k, v in item_count.items()}

    # Root node
    root = FPNode(None, 1, None)

    # Insert transactions
    for transaction in transactions:
        filtered = [item for item in transaction if item in item_count]
        filtered.sort(key=lambda x: item_count[x], reverse=True)

        current = root
        for item in filtered:
            if item in current.children:
                current.children[item].increment(1)
            else:
                new_node = FPNode(item, 1, current)
                current.children[item] = new_node

            # Update header table links
            if header_table[item][1] is None:
                header_table[item][1] = new_node
            else:
                node = header_table[item][1]
                while node.link:
                    node = node.link
                node.link = new_node

            current = current.children[item]

    return root, header_table
```

Find Prefix Paths

```
[4]: def find_prefix_paths(base_pat, node):
    cond_patterns = []

    while node:
        path = []
        parent = node.parent

        while parent and parent.item:
            path.append(parent.item)
            parent = parent.parent

        for _ in range(node.count):
            if path:
                cond_patterns.append(path)

        node = node.link

    return cond_patterns
```

Mine FP-Tree (FP-Growth)

```
[5]: def mine_fp_tree(header_table, min_support, prefix, freq_itemsets):
    sorted_items = sorted(header_table.items(), key=lambda x: x[1][0])

    for item, (count, node) in sorted_items:
        new_freq_set = prefix.copy()
        new_freq_set.add(item)

        freq_itemsets.append((new_freq_set, count))
        cond_patterns = find_prefix_paths(item, node)
        cond_tree, cond_header = build_fp_tree(cond_patterns, min_support)

        if cond_header:
            mine_fp_tree(cond_header, min_support, new_freq_set, freq_itemsets)
```

Main FP-Growth Function

```
[6]: def fp_growth(transactions, min_support):
    tree, header = build_fp_tree(transactions, min_support)
    freq_itemsets = []

    if header:
        mine_fp_tree(header, min_support, set(), freq_itemsets)

    return freq_itemsets
```

Load Dataset

```
[7]: import pandas as pd

df = pd.read_csv("dataset 7.csv")
transactions = df.values.tolist()
```

Run FP-Growth

```
[8]: min_support = 2

freq_itemsets = fp_growth(transactions, min_support)

print("Frequent Itemsets:\n")
for itemset, support in freq_itemsets:
    print(itemset, ":", support)
```

Frequent Itemsets:

```
{2} : 2
{'Shoes'} : 2
{4} : 2
{'Beverages'} : 2
{5} : 2
{6} : 2
{7} : 2
{8} : 2
{10} : 2
{1} : 3
{'Mobile'} : 3
{'Clothing'} : 3
{3} : 3
{'Groceries'} : 3
{9} : 3
{'Electronics'} : 4
{'Accessories'} : 5
```

Lab 8

Python Syntax & Basics

Objective

To understand basic Python syntax and fundamental programming concepts used in data mining and data analysis tasks.

1. Introduction

Python is one of the most widely used programming languages in data mining due to its simplicity, readability, and powerful libraries such as Pandas, NumPy, and Scikit-learn. It allows data scientists to efficiently perform data preprocessing, analysis, visualization, and model building.

In data mining, Python is mainly used to handle datasets, clean data, perform calculations, and apply machine learning algorithms.

2. Python Syntax Basics

2.1 Variables

- Variables are used to store data values.
- Python does not require explicit declaration of data types.

Importance in Data Mining

- Used to store dataset values, features, and model outputs.

2.2 Data Types

- Integer (int)
- Float (decimal values)
- String (text data)
- Boolean (True/False)

Importance in Data Mining

- Helps define type of dataset attributes.
- Used for classification and numerical analysis.

2.3 Lists and Arrays

- Lists store multiple values in a single variable.
- Similar to arrays in data processing.

Importance in Data Mining

- Used to store datasets, records, and feature values.

2.4 Conditional Statements

- if, elif, else statements are used for decision-making.

Importance in Data Mining

- Used for filtering data.
- Helps in classification logic.

2.5 Loops

- for loop and while loop are used for iteration.

Importance in Data Mining

- Used to process large datasets row by row.
- Helpful in feature extraction and preprocessing.

2.6 Functions

- Functions are reusable blocks of code.

Importance in Data Mining

- Used to build preprocessing pipelines.
- Helps modularize data processing steps.

3. Python Libraries for Data Mining

NumPy

- Used for numerical operations and arrays.

Pandas

- Used for data manipulation and dataset handling.

Matplotlib / Seaborn

- Used for data visualization.

Scikit-learn

- Used for machine learning algorithms.

Import Libraries

```
[1]: import pandas as pd
import numpy as np
```

Basic Variables and Data Types

```
[2]: a = 10
b = 3.5
name = "Data Mining"
flag = True
print(a, b, name, flag)

10 3.5 Data Mining True
```

Lists

```
[3]: items = ['milk', 'bread', 'butter']
items.append('jam')
items.remove('bread')
print(items)

['milk', 'butter', 'jam']
```

Dictionaries

```
[4]: freq = {'milk': 2, 'bread': 3}
freq['butter'] = 1
freq['milk'] += 1
print(freq)

{'milk': 3, 'bread': 3, 'butter': 1}
```

Loops

```
[5]: transactions = [
    ['milk', 'bread'],
    ['bread', 'butter'],
    ['milk', 'butter']
]
for t in transactions:
    print(t)

['milk', 'bread']
['bread', 'butter']
['milk', 'butter']
```

Conditional Statements

```
[6]: support = 2
if support >= 2:
    print("Frequent Item")
else:
    print("Not Frequent")

Frequent Item
```

Functions

```
[7]: def count_items(transactions):
    freq = {}

    for t in transactions:
        for item in t:
            freq[item] = freq.get(item, 0) + 1

    return freq

transactions = [
    ['milk', 'bread'],
    ['bread', 'butter'],
    ['milk', 'butter']
]

print(count_items(transactions))
{'milk': 2, 'bread': 2, 'butter': 2}
```

NumPy Basics

```
[8]: import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print("Mean:", np.mean(arr))
print("Sum:", np.sum(arr))

Mean: 3.0
Sum: 15
```

Pandas Basics

```
[9]: import pandas as pd

data = {
    'Item': ['milk', 'bread', 'butter'],
    'Sales': [10, 15, 7]
}

df = pd.DataFrame(data)

print(df)
```

	Item	Sales
0	milk	10
1	bread	15
2	butter	7

Reading Dataset

```
[10]: df = pd.read_csv("dataset 4.csv")
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	

Data Cleaning

```
[11]: df = df.dropna()
df = df.drop_duplicates()
print(df)
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	Urban
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	Urban
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	Urban
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	Urban
5	LP001054	Male	Yes	0	Not Graduate	Yes	2165	3422	152.0	360.0	1.0	Urban
...	...	...	...	...	...	...	...	...	...	...	...	...
361	LP002969	Male	Yes	1	Graduate	No	2269	2107	99.0	360.0	1.0	Semiurban
362	LP002971	Male	Yes	3+	Not Graduate	Yes	4009	1777	113.0	360.0	1.0	Urban
363	LP002975	Male	Yes	0	Graduate	No	4158	709	115.0	360.0	1.0	Urban
365	LP002986	Male	Yes	0	Graduate	No	5000	2393	158.0	360.0	1.0	Rural
366	LP002989	Male	No	0	Graduate	Yes	5200	0	58.0	180.0	1.0	Rural

Basic Data Analysis

```
[12]: print(df.describe())
      print(df['Gender'].value_counts())
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
count	289.000000	289.000000	289.000000	289.000000
mean	4637.352941	1528.262976	136.792388	342.671280
std	4790.683934	2377.599209	59.699582	65.655503
min	0.000000	0.000000	28.000000	6.000000
25%	2875.000000	0.000000	102.000000	360.000000
50%	3833.000000	879.000000	126.000000	360.000000
75%	5000.000000	2400.000000	158.000000	360.000000
max	72529.000000	24000.000000	460.000000	480.000000

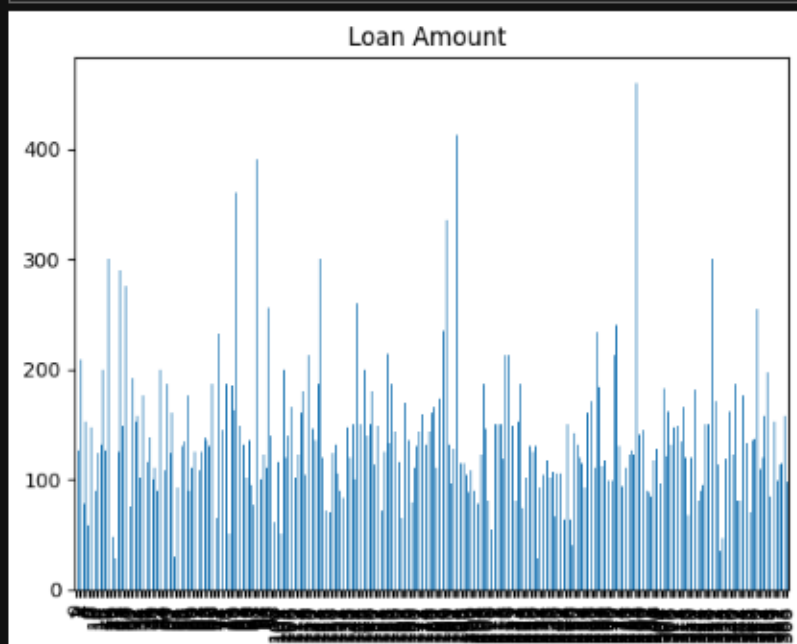
Credit_History	
count	289.000000
mean	0.840830
std	0.366469
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

Gender	
Male	230
Female	59

Name: count, dtype: int64

Visualization

```
[13]: import matplotlib.pyplot as plt
      df['LoanAmount'].plot(kind='bar')
      plt.title("Loan Amount ")
      plt.show()
```



Lab 9

Decision Tree Classifier

Objective

To understand the concept of Decision Tree Classifier and its use in solving classification problems in data mining and machine learning.

1. Introduction

A Decision Tree Classifier is a supervised machine learning algorithm used for classification tasks. It works by splitting data into branches based on feature values and making decisions in a tree-like structure. Each internal node represents a decision based on an attribute, and each leaf node represents a class label or final output.

Decision trees are widely used in data mining because they are easy to understand, interpret, and implement.

2. Structure of Decision Tree

Root Node

- The top node of the tree.
- Represents the entire dataset.

Decision Nodes

- Nodes where data is split based on a feature.

Leaf Nodes

- Final output nodes.
- Represent class labels or decisions.

3. Working of Decision Tree Classifier

Step 1: Select Best Attribute

- The algorithm selects the best feature to split the dataset.
- Selection is based on criteria like Gini Index or Information Gain.

Step 2: Split Dataset

- Data is divided into subsets based on selected attribute.

Step 3: Repeat Process

- Splitting continues recursively for each subset.

Step 4: Stop Condition

- Tree stops when:
 - All data belongs to one class.
 - No attributes are left.
 - Predefined depth is reached.

Step 1: Import Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)
from sklearn.preprocessing import LabelEncoder

import warnings
warnings.filterwarnings('ignore')

# Set style
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 120

print('✅ Libraries imported successfully!')
```

✅ Libraries imported successfully!

Step 2: Load & Explore the Dataset

```
[2]: # --- Generate synthetic Marketing Dataset ---
np.random.seed(42)
n = 500

age = np.random.randint(18, 70, n)
income = np.random.randint(20000, 120000, n)
gender = np.random.choice(['Male', 'Female'], n)
education = np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n)
marital = np.random.choice(['Single', 'Married', 'Divorced'], n)
campaign = np.random.randint(1, 6, n) # number of campaigns contacted
prev_purch = np.random.randint(0, 5, n) # previous purchases
web_visits = np.random.randint(0, 20, n) # website visits per month

# Simulate purchase probability based on features
prob = (
    0.3
    + 0.004 * (age - 18)
    + 0.00003 * income
    + 0.05 * prev_purch
    + 0.015 * web_visits
    - 0.02 * campaign
)
prob = np.clip(prob, 0.05, 0.95)
purchased = (np.random.rand(n) < prob).astype(int)

df = pd.DataFrame({
    'Age': age,
    'Income': income,
    'Gender': gender,
    'Education': education,
    'Marital_Status': marital,
    'Campaign_Contact': campaign,
    'Previous_Purchases': prev_purch,
    'Web_Visits': web_visits,
    'Purchased': purchased
})

print('Dataset Shape:', df.shape)
df.head(10)
```

Dataset Shape: (500, 9)

	Age	Income	Gender	Education	Marital_Status	Campaign_Contact	Previous_Purchases	Web_Visits	Purchased
0	56	23343	Male	High School	Married	3	0	0	0
1	69	33500	Male	Bachelor	Divorced	1	2	7	0
2	46	73222	Female	Master	Single	1	2	5	1
3	32	49375	Male	Master	Divorced	5	4	5	0
4	60	29662	Female	PhD	Divorced	1	3	17	1
5	25	36964	Female	Bachelor	Single	4	0	16	1
6	38	79638	Male	Master	Single	2	4	11	1
7	56	93666	Male	Bachelor	Divorced	5	1	5	1
8	36	105067	Male	Master	Single	3	1	7	1
9	40	87215	Female	High School	Single	4	0	18	1

```
# Basic info
print('--- Dataset Info ---')
df.info()
print('\n--- Missing Values ---')
print(df.isnull().sum())
print('\n--- Statistical Summary ---')
df.describe()
```

--- Dataset Info ---

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Age                  500 non-null    int32
1   Income               500 non-null    int32
2   Gender               500 non-null    str
3   Education            500 non-null    str
4   Marital_Status       500 non-null    str
5   Campaign_Contact     500 non-null    int32
6   Previous_Purchases   500 non-null    int32
7   Web_Visits           500 non-null    int32
8   Purchased            500 non-null    int64
dtypes: int32(5), int64(1), str(3)
memory usage: 34.9 KB
```

--- Missing Values ---

```
Age                  0
Income               0
Gender               0
Education            0
Marital_Status       0
Campaign_Contact     0
Previous_Purchases   0
Web_Visits           0
Purchased            0
dtype: int64
```

--- Statistical Summary ---

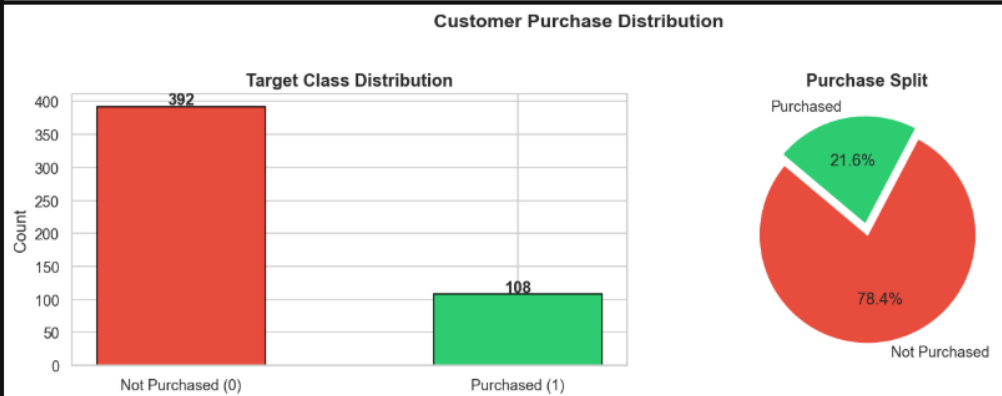
	Age	Income	Campaign_Contact	Previous_Purchases	Web_Visits	Purchased
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	44.220000	68167.952000	3.088000	2.032000	9.542000	0.784000
std	15.036082	28693.088647	1.431207	1.395303	5.739872	0.411926
min	18.000000	20055.000000	1.000000	0.000000	0.000000	0.000000
25%	32.000000	43369.750000	2.000000	1.000000	4.000000	1.000000
50%	45.000000	66636.000000	3.000000	2.000000	10.000000	1.000000
75%	57.000000	92395.250000	4.000000	3.000000	15.000000	1.000000
max	69.000000	119835.000000	5.000000	4.000000	19.000000	1.000000


```
# Target class distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

counts = df['Purchased'].value_counts()
axes[0].bar(['Not Purchased (0)', 'Purchased (1)'], counts.values,
            color=['#e74c3c', '#2ecc71'], edgecolor='black', width=0.5)
axes[0].set_title('Target Class Distribution', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Count')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 3, str(v), ha='center', fontweight='bold')

axes[1].pie(counts.values, labels=['Not Purchased', 'Purchased'],
            autopct='%1.1f%%', colors=['#e74c3c', '#2ecc71'],
            startangle=140, explode=(0.05, 0.05))
axes[1].set_title('Purchase Split', fontsize=13, fontweight='bold')

plt.suptitle('Customer Purchase Distribution', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

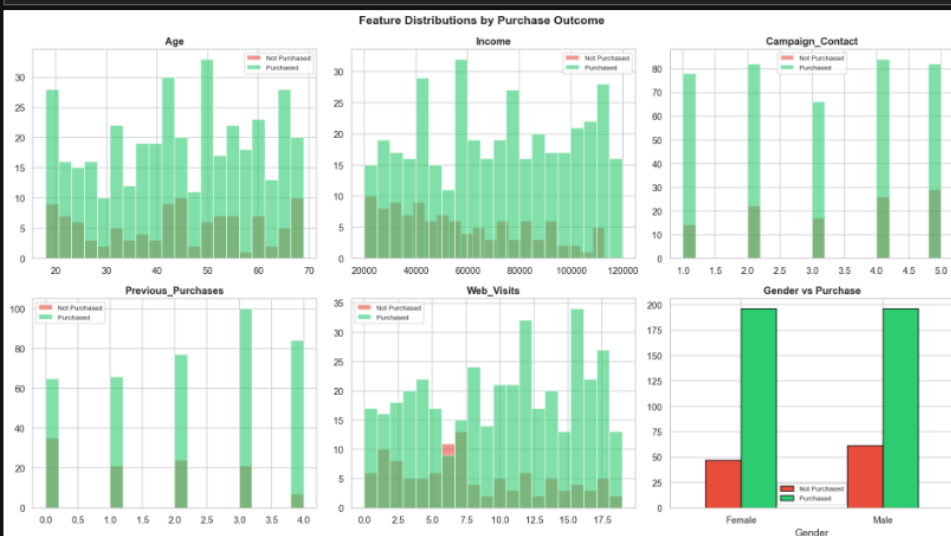


```
# Feature distributions by target
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
num_features = ['Age', 'Income', 'Campaign_Contact', 'Previous_Purchases', 'Web_Visits']

for ax, feat in zip(axes.flatten(), num_features):
    for val, color, label in zip([0, 1], ['#e74c3c', '#2ecc71'], ['Not Purchased', 'Purchased']):
        ax.hist(df[df['Purchased']==val][feat], bins=20, alpha=0.6,
                color=color, label=label, edgecolor='white')
    ax.set_title(feat, fontweight='bold')
    ax.legend(fontsize=8)

# Categorical - Gender
ax = axes[1][2]
cat_data = df.groupby(['Gender', 'Purchased']).size().unstack()
cat_data.plot(kind='bar', ax=ax, color=['#e74c3c', '#2ecc71'], edgecolor='black', rot=0)
ax.set_title('Gender vs Purchase', fontweight='bold')
ax.legend(['Not Purchased', 'Purchased'], fontsize=8)

plt.suptitle('Feature Distributions by Purchase Outcome', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Step 3: Data Preprocessing

```
# Encode categorical features
df_encoded = df.copy()
le = LabelEncoder()

cat_cols = ['Gender', 'Education', 'Marital_Status']
for col in cat_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
    print(f' {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}')

# Features & Target
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

feature_names = X.columns.tolist()

# Train-Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'\n✅ Preprocessing complete!')
print(f' Training samples : {X_train.shape[0]}')
print(f' Testing samples : {X_test.shape[0]}')
print(f' Features : {X_train.shape[1]}')

Gender: {'Female': np.int64(0), 'Male': np.int64(1)}
Education: {'Bachelor': np.int64(0), 'High School': np.int64(1), 'Master': np.int64(2), 'PhD': np.int64(3)}
Marital_Status: {'Divorced': np.int64(0), 'Married': np.int64(1), 'Single': np.int64(2)}

✅ Preprocessing complete!
Training samples : 400
Testing samples : 100
Features : 8
```

Step 4: Train the Decision Tree

```
# Train Decision Tree Classifier
dt_model = DecisionTreeClassifier(
    criterion = 'gini',          # Splitting criterion
    max_depth = 5,              # Limit depth to avoid overfitting
    min_samples_split = 10,
    min_samples_leaf = 5,
    random_state = 42
)

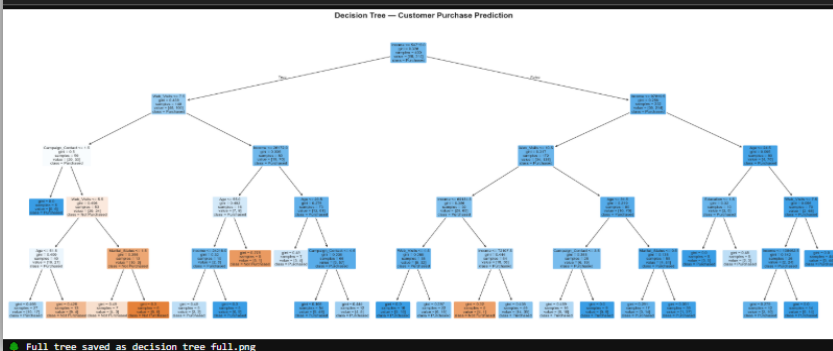
dt_model.fit(X_train, y_train)

print('✅ Decision Tree trained successfully!')
print(f' Tree Depth : {dt_model.get_depth()}')
print(f' Leaf Nodes : {dt_model.get_n_leaves()}')

✅ Decision Tree trained successfully!
Tree Depth : 5
Leaf Nodes : 24
```

Step 5: Visualize the Decision Tree

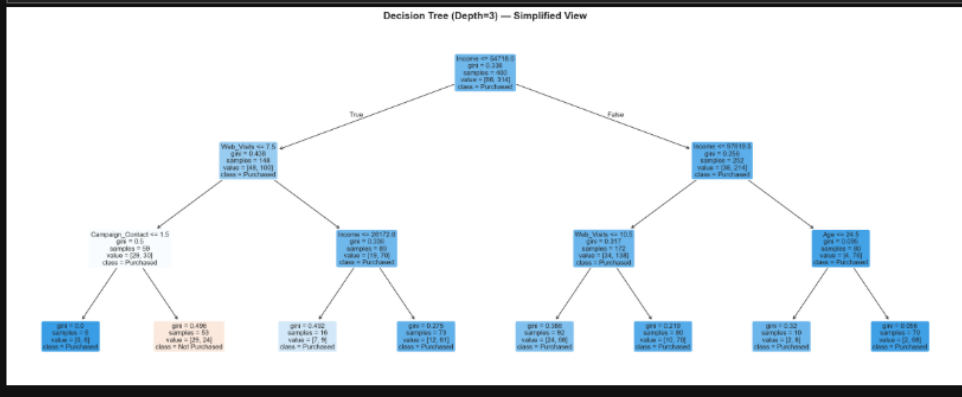
```
# Full tree visualization
fig, ax = plt.subplots(figsize=(24, 10))
plot_tree(
    dt_model,
    feature_names = feature_names,
    class_names = ['Not Purchased', 'Purchased'],
    filled = True,
    rounded = True,
    fontsize = 8,
    ax = ax,
    laparity = True,
    proportion = False
)
plt.title('Decision Tree - Customer Purchase Prediction', fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig('decision_tree_full.png', bbox_inches='tight', dpi=150)
plt.show()
print('✅ Full tree saved as decision_tree_full.png')
```



```
# Simplified view (depth=3) for readability
dt_simple = DecisionTreeClassifier(
    criterion='gini', max_depth=3, random_state=42
)
dt_simple.fit(X_train, y_train)

fig, ax = plt.subplots(figsize=(20, 8))
plot_tree(
    dt_simple,
    feature_names = feature_names,
    class_names    = ['Not Purchased', 'Purchased'],
    filled         = True,
    rounded        = True,
    fontsize       = 10,
    ax             = ax
)

plt.title('Decision Tree (Depth=3) - Simplified View', fontsize=15, fontweight='bold', pad=15)
plt.tight_layout()
plt.show()
print()
print('--- Text Representation ---')
print(export_text(dt_simple, feature_names=feature_names))
```



Step 6: Evaluate the Model

KPIs: Accuracy | Precision | Recall | F1-Score

```
[10]: # Predictions
y_pred = dt_model.predict(X_test)
y_pred_train = dt_model.predict(X_train)

# -- KPI Metrics --
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

train_acc = accuracy_score(y_train, y_pred_train)

print('=' * 50)
print('📊 MODEL PERFORMANCE (KPIs)')
print('=' * 50)
print(f'🎯 Accuracy : {accuracy:.4f} ({accuracy*100:.2f}%)')
print(f'🔍 Precision : {precision:.4f}')
print(f'📈 Recall : {recall:.4f}')
print(f'📊 F1-Score : {f1:.4f}')
print('=' * 50)
print(f'📈 Train Accuracy: {train_acc:.4f} ({train_acc*100:.2f}%)')
print(f'📊 OverFit Gap : {(train_acc - accuracy)*100:.2f}%)')
print('=' * 50)
```

```
=====
📊 MODEL PERFORMANCE (KPIs)
=====
🎯 Accuracy : 0.7280 (72.80%)
🔍 Precision : 0.7778
📈 Recall : 0.8974
📊 F1-Score : 0.8333
=====
📈 Train Accuracy: 0.8325 (83.25%)
📊 OverFit Gap : 11.25%
=====
```

```
[11]: # Detailed Classification Report
print('\n--- Classification Report ---')
print(classification_report(y_test, y_pred,
    target_names=['Not Purchased (0)', 'Purchased (1)']))
```

```
--- Classification Report ---
              precision    recall  f1-score   support

Not Purchased (0)    0.20      0.89      0.12         22
Purchased (1)        0.78      0.90      0.83         78

   accuracy            0.49
  macro avg            0.49      0.49      0.48         100
 weighted avg            0.65      0.72      0.68         100
```

```

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'],
            linewidths=1, linecolor='gray', ax=axes[0])
axes[0].set_title('Confusion Matrix', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Predicted Label', fontsize=11)
axes[0].set_ylabel('True Label', fontsize=11)

# KPI Bar Chart
kpi_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
kpi_values = [accuracy, precision, recall, f1]
bar_colors = ['#3498db', '#2ecc71', '#e67e22', '#9b59b6']

bars = axes[1].bar(kpi_names, kpi_values, color=bar_colors,
                  edgecolor='black', width=0.5)
axes[1].set_ylim(0, 1.15)
axes[1].set_title('KPI Summary', fontsize=13, fontweight='bold')
axes[1].set_ylabel('Score')
axes[1].axhline(y=0.8, color='red', linestyle='--', alpha=0.5, label='80% threshold')
axes[1].legend()

for bar, val in zip(bars, kpi_values):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                f'{val: .3f}', ha='center', fontweight='bold', fontsize=11)

plt.suptitle('Model Evaluation - Decision Tree', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



Step 7: Feature Importance

```

# Feature Importances
importances = pd.Series(dt_model.feature_importances_, index=feature_names)
importances = importances.sort_values(ascending=True)

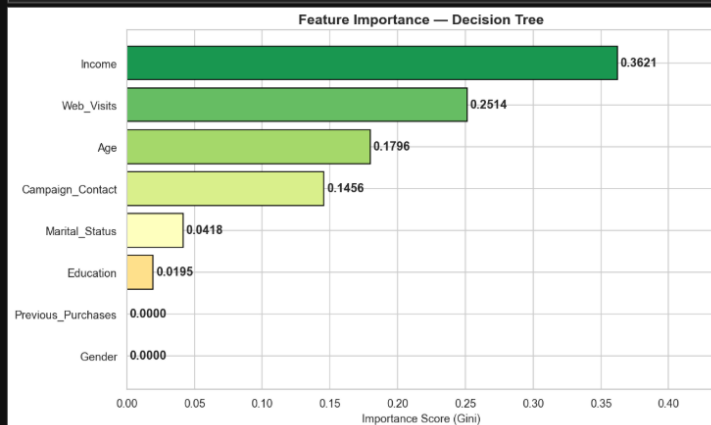
fig, ax = plt.subplots(figsize=(10, 6))
colors = plt.cm.RdYlGn(np.linspace(0.2, 0.9, len(importances)))
bars = ax.barh(importances.index, importances.values, color=colors, edgecolor='black')

for bar, val in zip(bars, importances.values):
    ax.text(val + 0.002, bar.get_y() + bar.get_height()/2,
            f'{val: .4f}', va='center', fontweight='bold')

ax.set_title('Feature Importance - Decision Tree', fontsize=14, fontweight='bold')
ax.set_ylabel('Importance Score (Gini)', fontsize=11)
ax.set_xlim(0, importances.max() * 1.2)
plt.tight_layout()
plt.show()

print("\n--- Feature Importance Ranking ---")
for i, (feat, imp) in enumerate(importances.sort_values(ascending=False).items(), 1):
    bar = f'█' * int(imp * 100)
    print(f"{i}. {feat: <22} {imp: .4f} {bar}")

```



Lab 10

Naïve Bayes & KNN

Objective

To understand Naïve Bayes and K-Nearest Neighbors (KNN) algorithms and their role in classification tasks in data mining and machine learning.

1. Introduction

Naïve Bayes and K-Nearest Neighbors (KNN) are two important supervised learning algorithms used for classification problems. Both are widely used in data mining due to their simplicity, efficiency, and effectiveness on different types of datasets.

Naïve Bayes is a probabilistic model based on Bayes' Theorem, while KNN is a distance-based algorithm that classifies data based on similarity.

2. Naïve Bayes Classifier

2.1 Concept

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that all features are independent of each other, which is called the “naïve” assumption.

2.2 Bayes Theorem

- It calculates the probability of a class given input features.
- Used for prediction based on likelihood and prior probability.

2.3 Working of Naïve Bayes

- Calculate prior probabilities for each class.
- Compute likelihood of features for each class.
- Apply Bayes' formula to find posterior probability.
- Assign class with highest probability.

3. K-Nearest Neighbors (KNN)

3.1 Concept

KNN is a simple, instance-based learning algorithm that classifies a data point based on the majority class of its nearest neighbors.

3.2 Working of KNN

- Choose the value of K (number of neighbors).
- Calculate distance between data points.
- Identify K nearest neighbors.
- Assign class based on majority vote.

3.3 Distance Measures

- Euclidean distance (most common)
- Manhattan distance
- Minkowski distance

1. Imports & Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, roc_curve, auc, f1_score
)
from sklearn.pipeline import Pipeline

# Optional: download dataset
# pip install datasets OR use UCI / Kaggle SMS Spam Collection

print('Libraries loaded successfully ✓')

Libraries loaded successfully ✓
```

2. Load Dataset

```
# — Option A: Load from local file (SMS Spam Collection, UCI) —————
df = pd.read_csv('spam.csv', encoding='latin-1')[['v1', 'v2']]
df.columns = ['label', 'text']

# — Option B: Download programmatically —————
url = 'https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv'
df = pd.read_table(url, header=None, names=['label', 'text'])

print(f'Shape: {df.shape}')
df.head()
```

Shape: (5572, 2)

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

3. Exploratory Data Analysis

```
print('Class distribution:')
print(df['label'].value_counts())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

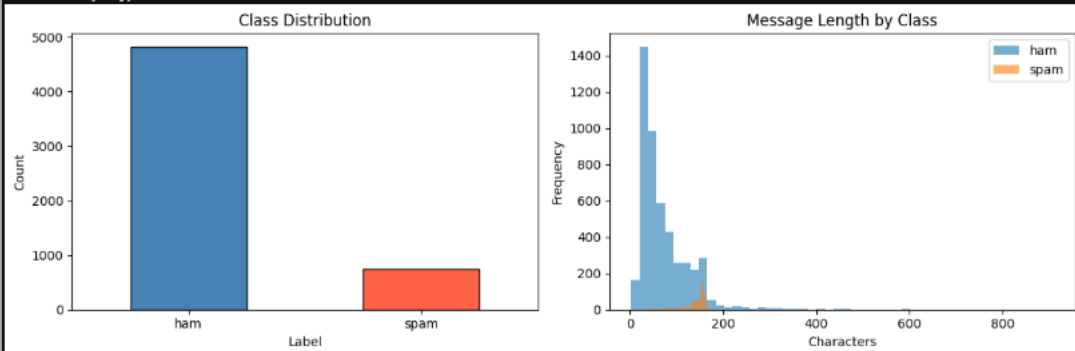
# Class balance
df['label'].value_counts().plot(kind='bar', ax=axes[0], color=['steelblue', 'tomato'], edgecolor='black')
axes[0].set_title('Class Distribution')
axes[0].set_xlabel('Label'); axes[0].set_ylabel('Count')
axes[0].tick_params(rotation=0)

# Message Length distribution
df['msg_len'] = df['text'].apply(len)
df.groupby('label')['msg_len'].plot(kind='hist', bins=50, alpha=0.6, ax=axes[1], legend=True)
axes[1].set_title('Message Length by Class')
axes[1].set_xlabel('Characters')

plt.tight_layout()
plt.show()

print('\nSample spam messages:')
df[df['label']=='spam']['text'].head(3).values
```

Class distribution:
label
ham 4825
spam 747
Name: count, dtype: int64



4. Preprocessing

```
le = LabelEncoder()
df['label_enc'] = le.fit_transform(df['label']) # ham=0, spam=1

X_text = df['text']
y = df['label_enc']

# -- TF-IDF Vectorization --
tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
X_tfidf = tfidf.fit_transform(X_text)

# -- Count Vectorization (for Multinomial NB) --
cvec = CountVectorizer(stop_words='english', max_features=5000)
X_count = cvec.fit_transform(X_text)

# Train / Test split
X_tr_t, X_te_t, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.2, random_state=42, stratify=y)
X_tr_c, X_te_c, _, _ = train_test_split(X_count, y, test_size=0.2, random_state=42, stratify=y)

print(f'Train size: {X_tr_t.shape[0]} | Test size: {X_te_t.shape[0]}')
Train size: 4457 | Test size: 1115
```

5. Model Training

```
# — 5a. Multinomial Naïve Bayes —————
mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_tr_c, y_train)
y_pred_mnb = mnb.predict(X_te_c)
y_prob_mnb = mnb.predict_proba(X_te_c)[:, 1]

# — 5b. Gaussian Naïve Bayes (dense) —————
gnb = GaussianNB()
gnb.fit(X_tr_t.toarray(), y_train)
y_pred_gnb = gnb.predict(X_te_t.toarray())
y_prob_gnb = gnb.predict_proba(X_te_t.toarray())[:, 1]

# — 5c. KNN —————
# Find best k
k_range = range(1, 21)
k_scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k, metric='cosine')
    score = cross_val_score(knn, X_tr_t, y_train, cv=5, scoring='accuracy').mean()
    k_scores.append(score)

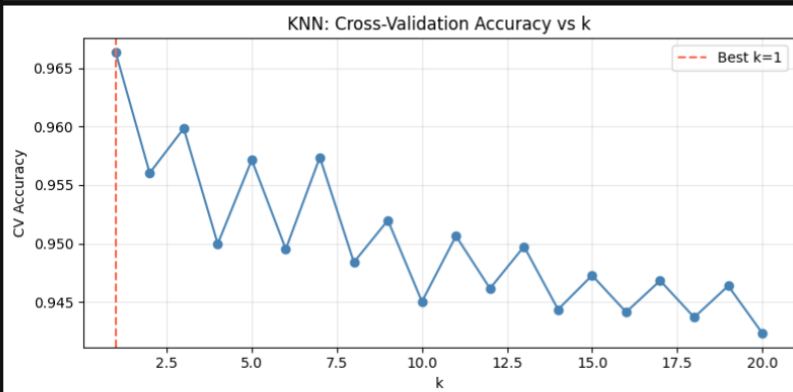
best_k = k_range[np.argmax(k_scores)]
print(f'Best k = {best_k} (CV accuracy = {max(k_scores):.4f})')

knn = KNeighborsClassifier(n_neighbors=best_k, metric='cosine')
knn.fit(X_tr_t, y_train)
y_pred_knn = knn.predict(X_te_t)
y_prob_knn = knn.predict_proba(X_te_t)[:, 1]

print('All models trained ✓')

Best k = 1 (CV accuracy = 0.9663)
All models trained ✓
```

```
plt.figure(figsize=(8, 4))
plt.plot(k_range, k_scores, marker='o', color='steelblue')
plt.axvline(best_k, color='tomato', linestyle='--', label=f'Best k={best_k}')
plt.title('KNN: Cross-Validation Accuracy vs k')
plt.xlabel('k'); plt.ylabel('CV Accuracy')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```



6. KPI Evaluation — Accuracy & False Positive Rate

```
def false_positive_rate(y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()
    return FP / (FP + TN)

models = {
    'Multinomial NB': (y_pred_mnb, y_prob_mnb),
    'Gaussian NB': (y_pred_gnb, y_prob_gnb),
    f'KNN (k={best_k})': (y_pred_knn, y_prob_knn),
}

results = []
for name, (y_pred, _) in models.items():
    acc = accuracy_score(y_test, y_pred)
    fpr = false_positive_rate(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results.append({'Model': name, 'Accuracy': acc, 'False Positive Rate': fpr, 'F1-Score (Spam)': f1})

res_df = pd.DataFrame(results).set_index('Model')
print(res_df.to_string())
```

Model	Accuracy	False Positive Rate	F1-Score (Spam)
Multinomial NB	0.984753	0.007246	0.942373
Gaussian NB	0.870852	0.131470	0.647059
KNN (k=1)	0.956054	0.019669	0.829268

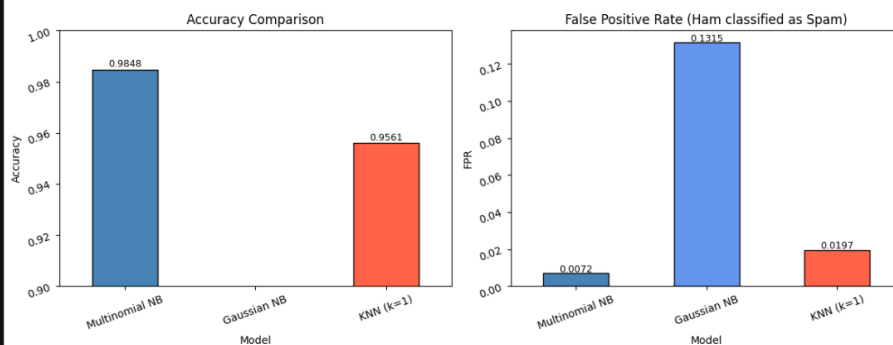
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

res_df['Accuracy'].plot(kind='bar', ax=axes[0], color=['steelblue', 'cornflowerblue', 'tomato'], edgecolor='black')
axes[0].set_title('Accuracy Comparison'); axes[0].set_ylim(0.9, 1.0)
axes[0].set_ylabel('Accuracy'); axes[0].tick_params(rotation=20)
for p in axes[0].patches:
    axes[0].annotate(f'{p.get_height():.4f}', (p.get_x()+p.get_width()/2, p.get_height()+0.001), ha='center', fontsize=9)

res_df['False Positive Rate'].plot(kind='bar', ax=axes[1], color=['steelblue', 'cornflowerblue', 'tomato'], edgecolor='black')
axes[1].set_title('False Positive Rate (Ham classified as Spam)'); axes[1].set_ylim(0.0, 0.14)
axes[1].set_ylabel('FPR'); axes[1].tick_params(rotation=20)
for p in axes[1].patches:
    axes[1].annotate(f'{p.get_height():.4f}', (p.get_x()+p.get_width()/2, p.get_height()+0.0005), ha='center', fontsize=9)

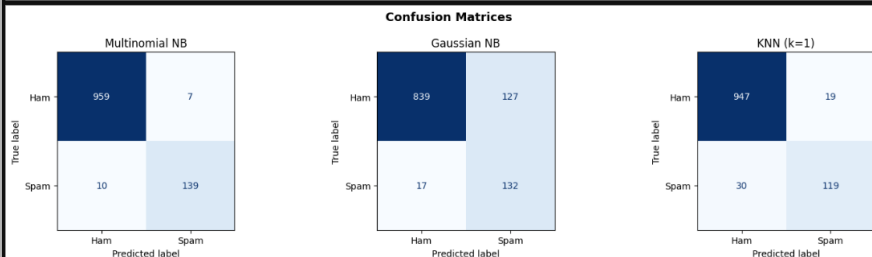
plt.suptitle('Lab 10 KPI Comparison — NB vs KNN', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```

Lab 10 KPI Comparison — NB vs KNN



7. Confusion Matrices ¶

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (name, y_pred, _) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=['Ham', 'Spam'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(name)
plt.suptitle('Confusion Matrices', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()
```



Lab 11

Support Vector Machine (SVM)

Objective

To understand the concept of Support Vector Machine (SVM) and its application in classification problems in data mining and machine learning.

1. Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. In data mining, it is mainly used for classification problems. SVM works by finding the best decision boundary (hyperplane) that separates different classes in the dataset.

The main idea of SVM is to maximize the margin between different classes so that the model becomes more accurate and generalizes well on unseen data.

2. Working of SVM

Step 1: Plot Data Points

- Data points are plotted in an n-dimensional space based on features.

Step 2: Find Hyperplane

- A hyperplane is selected to separate different classes.

Step 3: Maximize Margin

- SVM chooses the hyperplane with the maximum distance between classes.
- These boundary points are called support vectors.

Step 4: Classification

- New data points are classified based on which side of the hyperplane they fall.

3. Key Concepts

Hyperplane

- A decision boundary that separates different classes.

Support Vectors

- Data points closest to the hyperplane.
- They influence the position of the hyperplane.

Margin

- The distance between the hyperplane and support vectors.
- Larger margin improves model performance.

4. Types of SVM

Linear SVM

- Used when data is linearly separable.

Non-Linear SVM

- Used when data is not linearly separable.
- Uses kernel functions to transform data.

1. Imports & Setup

```
!pip install imbalanced-learn

Defaulting to user installation because normal site-packages is not writeable
Collecting imbalanced-learn
  Downloading imbalanced_learn-0.14.2-py3-none-any.whl.metadata (8.9 kB)
Requirement already satisfied: numpy<3,>=1.25.2 in C:\Users\Mishal nadeem\AppData\Roaming\Python\Python314\site-packages (from imbalanced-learn) (2.4.4)
Requirement already satisfied: scipy<2,>=1.11.4 in C:\Users\Mishal nadeem\AppData\Roaming\Python\Python314\site-packages (from imbalanced-learn) (1.17.1)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in C:\Users\Mishal nadeem\AppData\Roaming\Python\Python314\site-packages (from imbalanced-learn) (1.8.0)
Collecting sklearn-compat<0.2,>=0.1.6 (from imbalanced-learn)
  Downloading sklearn_compat-0.1.6-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: joblib<2,>=1.2.0 in C:\Users\Mishal nadeem\AppData\Roaming\Python\Python314\site-packages (from imbalanced-learn) (1.5.3)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in C:\Users\Mishal nadeem\AppData\Roaming\Python\Python314\site-packages (from imbalanced-learn) (3.6.0)
Downloading imbalanced_learn-0.14.2-py3-none-any.whl (236 kB)
Downloading sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Installing collected packages: sklearn-compat, imbalanced-learn
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, roc_curve, auc,
    precision_score, recall_score, f1_score, roc_auc_score
)
from sklearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

print('Libraries loaded ✓')
Libraries loaded ✓
```

2. Load Dataset

```
# -- Option A: Kaggle Credit Card Fraud Dataset -----
# df = pd.read_csv('creditcard.csv')

# -- Option B: Synthetic fraud dataset (reproducible demo) -----
from sklearn.datasets import make_classification

X_raw, y_raw = make_classification(
    n_samples=10000,
    n_features=29,
    n_informative=15,
    n_redundant=5,
    n_clusters_per_class=2,
    weights=[0.98, 0.02], # ~2 % fraud - realistic imbalance
    flip_y=0,
    random_state=42
)

feature_cols = [f'V{i}' for i in range(1, 29)] + ['Amount']
df = pd.DataFrame(X_raw, columns=feature_cols)
df['Class'] = y_raw

print(f'Shape: {df.shape}')
print('Class distribution:')
print(df['Class'].value_counts())
df.head()
```

Shape: (10000, 30)

Class distribution:

Class

0 9800

1 200

Name: count, dtype: int64

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V21	V22	V23	V24	V25
0	1.472217	-3.406766	-1.320317	-0.061705	-0.490147	-1.494240	0.081637	1.442300	-2.719120	-0.106241	...	-1.716629	-1.256745	-1.342324	-1.424618	4.97324
1	1.231409	-1.827861	0.082641	-0.141966	2.415254	-1.726895	-0.289643	-1.779067	2.934473	1.715699	...	3.220590	0.048269	-0.258278	2.505909	2.70541
2	-2.389706	0.644914	1.012999	2.083017	-1.044093	0.836513	0.432350	-5.806734	-3.280303	-0.764506	...	-0.982274	5.797412	-1.210273	-4.298172	4.09201
3	4.794456	-2.844959	-3.351036	-0.786301	-0.190488	-0.568891	1.366282	-1.350251	-3.170695	0.270654	...	-1.943101	1.715966	-0.164909	-0.578103	4.53481
4	-2.510845	1.025976	-0.659881	-0.882472	0.405543	2.051268	1.639206	-2.403113	-4.209283	-0.768739	...	0.432290	1.356674	0.298697	-1.694040	-1.73791

5 rows x 30 columns

3. Exploratory Data Analysis

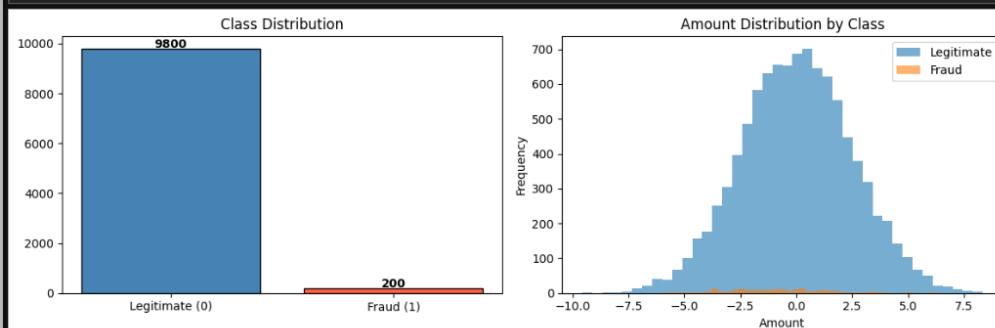
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Class imbalance
counts = df['Class'].value_counts()
axes[0].bar(['Legitimate (0)', 'Fraud (1)'], counts.values, color=['steelblue', 'tomato'], edgecolor='black')
axes[0].set_title('Class Distribution')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 20, str(v), ha='center', fontweight='bold')

# Amount distribution by class
df.groupby('Class')['Amount'].plot(kind='hist', bins=40, alpha=0.6, ax=axes[1], legend=True)
axes[1].set_title('Amount Distribution by Class')
axes[1].set_xlabel('Amount')
axes[1].legend(['Legitimate', 'Fraud'])

plt.tight_layout(); plt.show()

print(f'\nFraud rate: {df["Class"].mean()*100:.2f} %')
```



Fraud rate: 2.00 %

4. Preprocessing & SMOTE Oversampling ¶

```
X = df.drop('Class', axis=1)
y = df['Class']

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train / Test split (stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Apply SMOTE to training set only
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print(f'Original train class counts: {dict(zip(np.unique(y_train, return_counts=True)))}')
print(f'After SMOTE: {dict(zip(np.unique(y_train_res, return_counts=True)))}')
print(f'Test set size: {X_test.shape[0]}')

Original train class counts: {np.int64(0): np.int64(7840), np.int64(1): np.int64(160)}
After SMOTE: {np.int64(0): np.int64(7840), np.int64(1): np.int64(7840)}
Test set size: 2000
```

5. Train SVM — Linear & RBF Kernels

```
# — 5a. Linear SVM —
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_res, y_train_res)
y_pred_lin = svm_linear.predict(X_test)
y_prob_lin = svm_linear.predict_proba(X_test)[:, 1]

# — 5b. RBF SVM —
svm_rbf = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
svm_rbf.fit(X_train_res, y_train_res)
y_pred_rbf = svm_rbf.predict(X_test)
y_prob_rbf = svm_rbf.predict_proba(X_test)[:, 1]

print('SVM (Linear) trained ✓')
print('SVM (RBF) trained ✓')

SVM (Linear) trained ✓
SVM (RBF) trained ✓
```

6. KPI Evaluation — ROC-AUC, Recall, Precision

```
svm_models = {
    'SVM - Linear': (y_pred_lin, y_prob_lin),
    'SVM - RBF' : (y_pred_rbf, y_prob_rbf),
}

rows = []
for name, (y_pred, y_prob) in svm_models.items():
    rows.append({
        'Model' : name,
        'ROC-AUC' : roc_auc_score(y_test, y_prob),
        'Recall (Fraud)': recall_score(y_test, y_pred),
        'Precision' : precision_score(y_test, y_pred),
        'F1-Score' : f1_score(y_test, y_pred),
        'Accuracy' : accuracy_score(y_test, y_pred),
    })

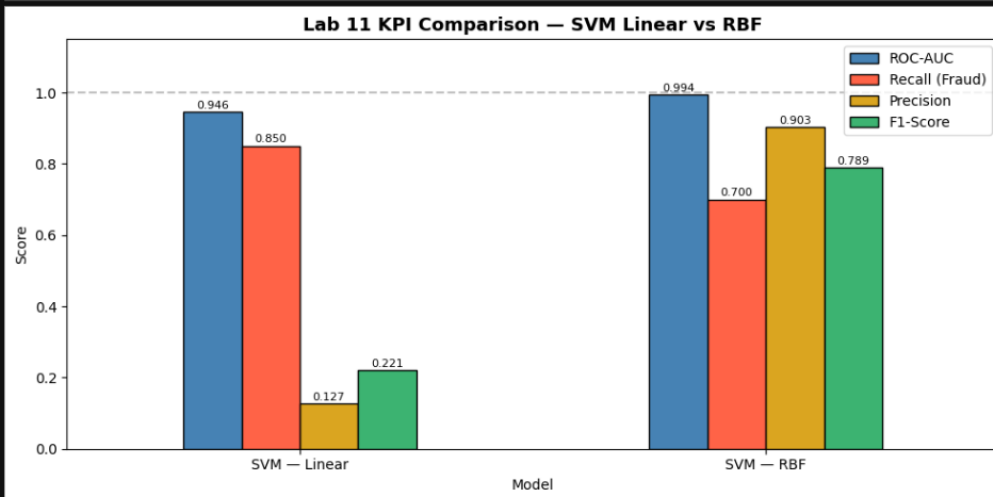
kpi_df = pd.DataFrame(rows).set_index('Model')
print(kpi_df.to_string())
```

	ROC-AUC	Recall (Fraud)	Precision	F1-Score	Accuracy
Model					
SVM - Linear	0.945753	0.85	0.126866	0.220779	0.8800
SVM - RBF	0.994375	0.70	0.903226	0.788732	0.9925

```

kpi_plot = kpi_df[['ROC-AUC', 'Recall (Fraud)', 'Precision', 'F1-Score']]
ax = kpi_plot.plot(kind='bar', figsize=(10, 5), edgecolor='black',
                  color=['steelblue', 'tomato', 'goldenrod', 'mediumseagreen'])
ax.set_title('Lab 11 KPI Comparison - SVM Linear vs RBF', fontsize=13, fontweight='bold')
ax.set_ylabel('Score'); ax.set_ylim(0, 1.15)
ax.tick_params(rotation=0); ax.legend(loc='upper right')
ax.axhline(1.0, color='gray', linestyle='--', alpha=0.5)
for p in ax.patches:
    ax.annotate(f'{p.get_height():.3f}',
                (p.get_x() + p.get_width()/2, p.get_height() + 0.01),
                ha='center', fontsize=8)
plt.tight_layout(); plt.show()

```

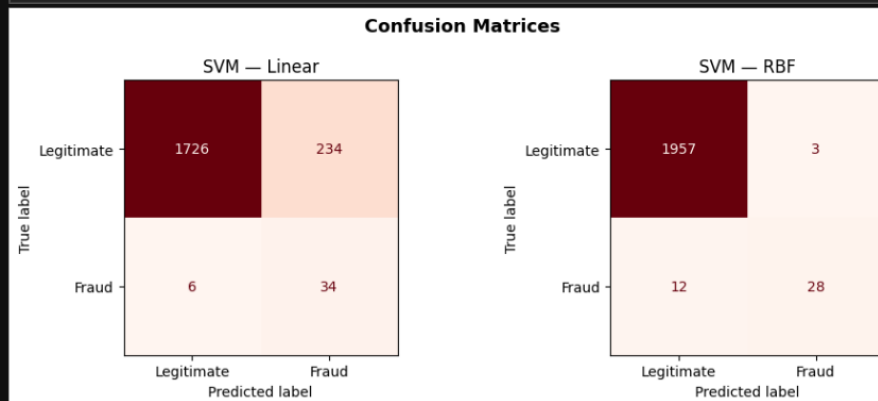


8. Confusion Matrices

```

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, (name, (y_pred, _)) in zip(axes, svm_models.items()):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=['Legitimate', 'Fraud'])
    disp.plot(ax=ax, colorbar=False, cmap='Reds')
    ax.set_title(name)
plt.suptitle('Confusion Matrices', fontsize=13, fontweight='bold')
plt.tight_layout(); plt.show()

```



Lab 12

K-Means & Hierarchical Clustering

Objective

To understand clustering techniques, specifically K-Means and Hierarchical Clustering, used for grouping similar data points in data mining.

1. Introduction

Clustering is an unsupervised learning technique used in data mining to group similar data points together based on their characteristics. Unlike classification, clustering does not require predefined labels. It helps in discovering hidden patterns and structures in data.

K-Means and Hierarchical Clustering are two widely used clustering algorithms.

2. K-Means Clustering

2.1 Concept

K-Means is a centroid-based clustering algorithm that divides data into K clusters based on similarity.

2.2 Working of K-Means

Step 1: Choose K

- Select the number of clusters (K).

Step 2: Initialize Centroids

- Randomly select K points as initial cluster centers.

Step 3: Assign Clusters

- Assign each data point to the nearest centroid.

Step 4: Update Centroids

- Recalculate centroids based on assigned points.

Step 5: Repeat

- Repeat steps until centroids no longer change.

3. Hierarchical Clustering

3.1 Concept

Hierarchical Clustering builds a tree-like structure (dendrogram) to represent nested clusters.

3.2 Types

Agglomerative (Bottom-Up)

- Each data point starts as a single cluster.
- Clusters are merged step by step.

Divisive (Top-Down)

- All data starts in one cluster.
- It is divided into smaller clusters.

3.3 Working of Hierarchical Clustering

- Calculate distance between data points.
- Merge or split clusters based on similarity.
- Continue until stopping condition is met.
- Represent results using a dendrogram.

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage

plt.rcParams['figure.figsize'] = (10, 6)
sns.set_style('whitegrid')
print('Libraries loaded successfully!')

Libraries loaded successfully!
```


2. Load & Explore Dataset

```
# --- Synthetic Mall Customer Dataset ---
np.random.seed(42)
n = 200
data = {
    'CustomerID': range(1, n+1),
    'Gender': np.random.choice(['Male', 'Female'], n),
    'Age': np.random.randint(18, 70, n),
    'Annual_Income_k': np.random.randint(15, 140, n),
    'Spending_Score': np.random.randint(1, 100, n)
}
df = pd.DataFrame(data)

# Inject cluster-like structure
for i in range(0, 40): df.loc[i, ['Annual_Income_k', 'Spending_Score']] = [20+np.random.randint(-5,5), 85+np.random.randint(-5,5)]
for i in range(40, 80): df.loc[i, ['Annual_Income_k', 'Spending_Score']] = [80+np.random.randint(-5,5), 80+np.random.randint(-5,5)]
for i in range(80, 120): df.loc[i, ['Annual_Income_k', 'Spending_Score']] = [50+np.random.randint(-5,5), 50+np.random.randint(-5,5)]
for i in range(120, 160): df.loc[i, ['Annual_Income_k', 'Spending_Score']] = [20+np.random.randint(-5,5), 20+np.random.randint(-5,5)]
for i in range(160, 200): df.loc[i, ['Annual_Income_k', 'Spending_Score']] = [85+np.random.randint(-5,5), 20+np.random.randint(-5,5)]

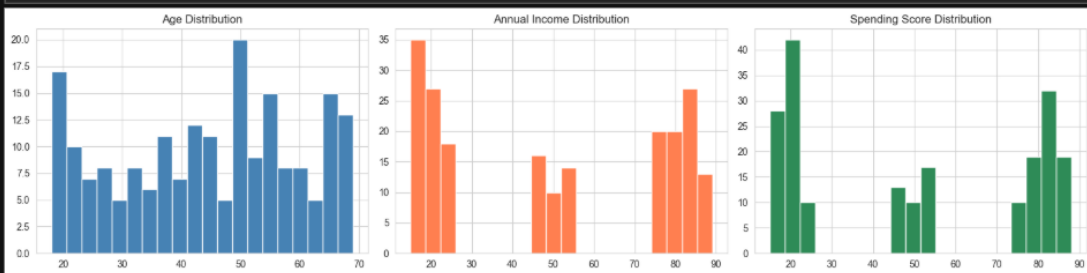
print('Shape:', df.shape)
df.head(10)
```

Shape: (200, 5)

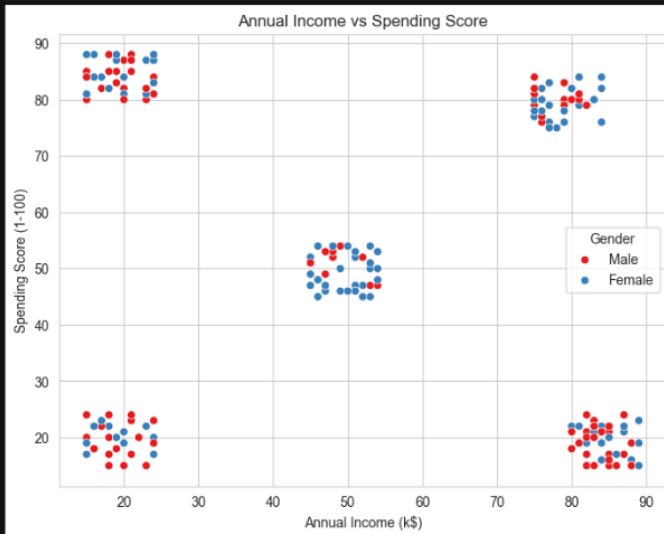
	CustomerID	Gender	Age	Annual_Income_k	Spending_Score
0	1	Male	49	23	80
1	2	Female	56	23	87
2	3	Male	66	15	85
3	4	Male	69	19	85
4	5	Male	49	24	84
5	6	Female	21	20	84
6	7	Male	47	19	83
7	8	Male	54	17	82
8	9	Male	40	18	88
9	10	Female	56	16	88

3. Exploratory Data Analysis ¶

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].hist(df['Age'], bins=20, color='steelblue', edgecolor='white')
axes[0].set_title('Age Distribution')
axes[1].hist(df['Annual_Income_k'], bins=20, color='coral', edgecolor='white')
axes[1].set_title('Annual Income Distribution')
axes[2].hist(df['Spending_Score'], bins=20, color='seagreen', edgecolor='white')
axes[2].set_title('Spending Score Distribution')
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Annual_Income_k', y='Spending_Score', hue='Gender', palette='Set1')
plt.title('Annual Income vs Spending Score')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.show()
```



5. K-Means Clustering

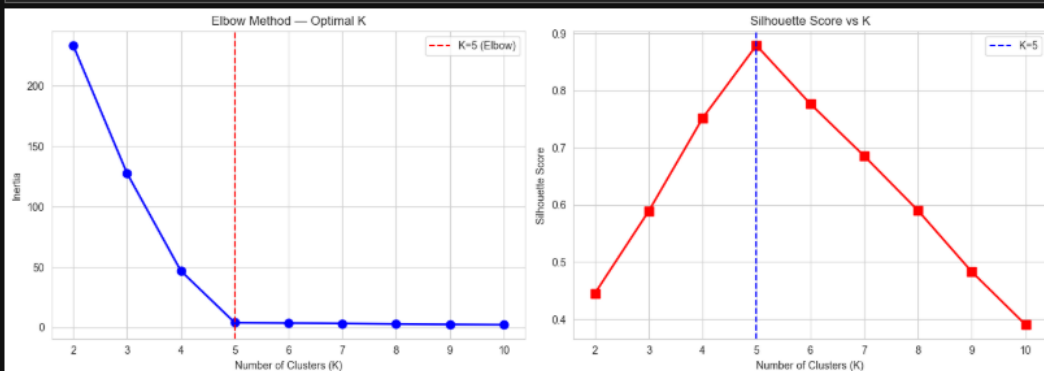
```
# Elbow Method
inertias = []
silhouettes = []
K_range = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, km.labels_))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(K_range, inertias, 'bo-', linewidth=2, markersize=8)
axes[0].set_title('Elbow Method - Optimal K')
axes[0].set_xlabel('Number of Clusters (K)')
axes[0].set_ylabel('Inertia')
axes[0].axvline(x=5, color='red', linestyle='--', label='K=5 (Elbow)')
axes[0].legend()

axes[1].plot(K_range, silhouettes, 'rs-', linewidth=2, markersize=8)
axes[1].set_title('Silhouette Score vs K')
axes[1].set_xlabel('Number of Clusters (K)')
axes[1].set_ylabel('Silhouette Score')
axes[1].axvline(x=5, color='blue', linestyle='--', label='K=5')
axes[1].legend()

plt.tight_layout()
plt.show()
print(f'Best Silhouette Score: {max(silhouettes):.4f} at K={K_range[silhouettes.index(max(silhouettes))]}')
```



```

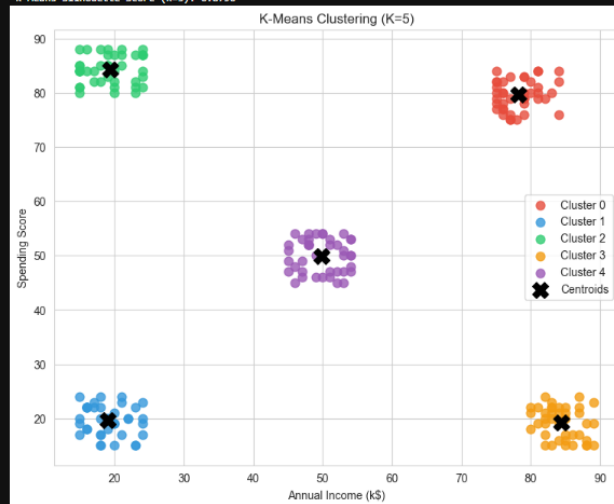
kmeans_sil = silhouette_score(X_scaled, df['KMeans_Cluster'])
print(f'K-Means Silhouette Score (K={optimal_k}): {kmeans_sil:.4f}')

# Visualize
plt.figure(figsize=(9, 7))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12', '#9b59b6']
for cluster in range(optimal_k):
    mask = df['KMeans_Cluster'] == cluster
    plt.scatter(df.loc[mask, 'Annual_Income_k'], df.loc[mask, 'Spending_Score'],
                c=colors[cluster], label=f'Cluster {cluster}', s=60, alpha=0.8)

centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(centers[:,0], centers[:,1], c='black', markers='x', s=200, zorder=5, label='Centroids')
plt.title('K-Means Clustering (K=5)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score')
plt.legend()
plt.show()

```

K-Means Silhouette Score (K=5): 0.8798

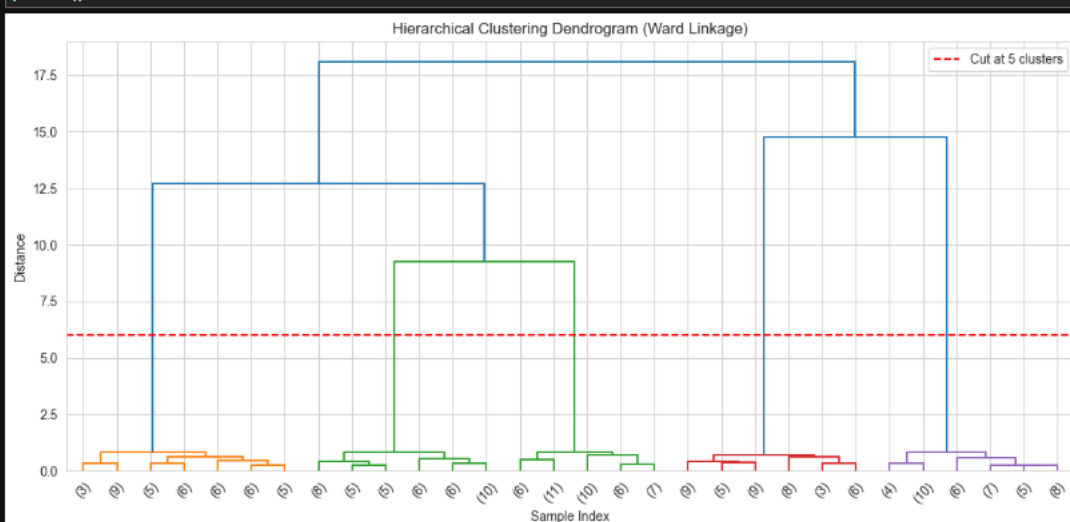


6. Hierarchical Clustering

```

# Dendrogram
plt.figure(figsize=(14, 6))
Z = linkage(X_scaled, method='ward')
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=45, leaf_font_size=10)
plt.title('Hierarchical Clustering Dendrogram (Ward Linkage)')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.axhline(y=6, color='red', linestyle='--', label='Cut at 5 clusters')
plt.legend()
plt.show()

```

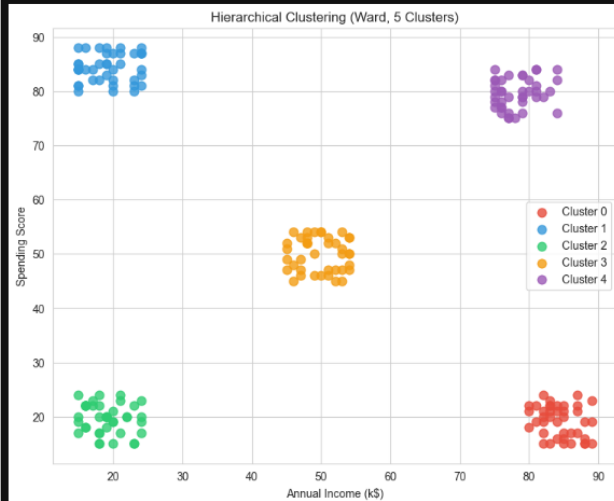


```
# Agglomerative Clustering
agglo = AgglomerativeClustering(n_clusters=5, linkage='ward')
df['Hierarchical_Cluster'] = agglo.fit_predict(X_scaled)

hier_sil = silhouette_score(X_scaled, df['Hierarchical_Cluster'])
print(f'Hierarchical Silhouette Score: {hier_sil:.4f}')

plt.figure(figsize=(9, 7))
for cluster in range(5):
    mask = df['Hierarchical_Cluster'] == cluster
    plt.scatter(df.loc[mask, 'Annual_Income_k'], df.loc[mask, 'Spending_Score'],
                c=colors[cluster], label=f'Cluster {cluster}', s=60, alpha=0.8)
plt.title('Hierarchical Clustering (Ward, 5 Clusters)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score')
plt.legend()
plt.show()

Hierarchical Silhouette Score: 0.8790
```



7. KPI — Cluster Revenue Contribution

```
cluster_labels = {0: 'Low Income / High Spend', 1: 'High Income / High Spend',
                  2: 'Average Income / Average Spend', 3: 'Low Income / Low Spend', 4: 'High Income / Low Spend'}

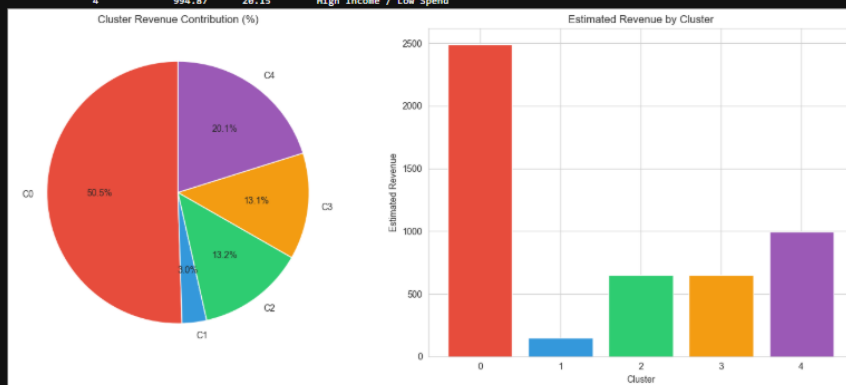
df['Estimated_Revenue'] = df['Annual_Income_k'] * df['Spending_Score'] / 100

revenue_by_cluster = df.groupby('KMeans_Cluster')['Estimated_Revenue'].sum().reset_index()
revenue_by_cluster['Revenue_%'] = (revenue_by_cluster['Estimated_Revenue'] /
                                   revenue_by_cluster['Estimated_Revenue'].sum() * 100).round(2)
revenue_by_cluster['Segment'] = revenue_by_cluster['KMeans_Cluster'].map(cluster_labels)
print(revenue_by_cluster.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].pie(revenue_by_cluster['Revenue_%'], labels=[f'C{c}' for c in revenue_by_cluster['KMeans_Cluster']],
            autopct='%1.1f%%', colors=colors, startangle=90)
axes[0].set_title('Cluster Revenue Contribution (%)')

axes[1].bar(revenue_by_cluster['KMeans_Cluster'], revenue_by_cluster['Estimated_Revenue'],
            color=colors, edgecolor='white')
axes[1].set_title('Estimated Revenue by Cluster')
axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('Estimated Revenue')
plt.tight_layout()
plt.show()
```

KMeans_Cluster	Estimated_Revenue	Revenue_%	Segment
0	2492.87	50.49	Low Income / High Spend
1	149.47	3.03	High Income / High Spend
2	652.34	13.21	Average Income / Average Spend
3	647.62	13.12	Low Income / Low Spend
4	994.87	20.15	High Income / Low Spend



Lab 13

Outlier Detection

Objective

To understand the concept of outliers in data mining and learn different techniques used to detect and handle outliers in datasets.

1. Introduction

Outliers are data points that significantly differ from the rest of the dataset. They may be much higher or lower than normal values and can distort analysis results if not handled properly. Outlier detection is an important step in data preprocessing because it improves data quality and ensures accurate model performance.

Outliers can occur due to errors in data collection, measurement issues, or genuine rare events.

2. Types of Outliers

2.1 Point Outliers

- A single data point that is significantly different from other values.
- Example: A salary of 1,000,000 in a dataset where most salaries are 30,000–50,000.

2.2 Contextual Outliers

- A data point that is abnormal in a specific context.
- Example: High temperature in winter season.

2.3 Collective Outliers

- A group of data points that behave unusually compared to the rest of the dataset.

3. Causes of Outliers

- Data entry errors.
- Measurement errors.
- Equipment malfunction.
- Fraudulent activity.
- Natural variation in data.

4. Techniques for Outlier Detection

4.1 Statistical Methods

- Use mean, median, standard deviation.
- Z-score method to detect extreme values.

4.2 Visualization Methods

- Box plots.
- Scatter plots.
- Histograms.

4.3 Distance-Based Methods

- Identify points far away from clusters.
- Used in KNN-based approaches.

4.4 Clustering-Based Methods

- Outliers do not belong to any cluster or form very small clusters.

4.5 Machine Learning Methods

- Isolation Forest.
- Local Outlier Factor (LOF).

5. Handling Outliers

- Remove outliers from dataset.
- Cap or trim extreme values.
- Transform data using log or scaling.
- Replace with mean or median values.

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from scipy.spatial.distance import mahalanobis
import scipy.stats as stats

plt.rcParams['figure.figsize'] = (10, 6)
sns.set_style('whitegrid')
print('Libraries loaded!')
```

Libraries loaded!

2. Dataset

```

np.random.seed(42)

# Normal transactions
n_normal = 950
normal = pd.DataFrame({
    'transaction_id': range(1, n_normal+1),
    'amount': np.random.lognormal(mean=4.5, sigma=0.8, size=n_normal),
    'frequency_per_day': np.random.poisson(lam=3, size=n_normal),
    'hour_of_day': np.random.randint(5, 23, size=n_normal),
    'distance_from_home_km': np.abs(np.random.normal(20, 15, n_normal)),
    'is_fraud': 0
})

# Fraudulent transactions
n_fraud = 50
fraud = pd.DataFrame({
    'transaction_id': range(n_normal+1, n_normal+n_fraud+1),
    'amount': np.random.uniform(5000, 20000, n_fraud),
    'frequency_per_day': np.random.randint(15, 40, n_fraud),
    'hour_of_day': np.random.choice([0, 1, 2, 3, 4], n_fraud),
    'distance_from_home_km': np.random.uniform(500, 2000, n_fraud),
    'is_fraud': 1
})

df = pd.concat([normal, fraud], ignore_index=True).sample(frac=1, random_state=42).reset_index(drop=True)

print(f'Dataset shape: {df.shape}')
print(f'Fraud ratio: {df["is_fraud"].mean()*100:.1f}%')
df.head()

```

Dataset shape: (1000, 6)
Fraud ratio: 5.0%

	transaction_id	amount	frequency_per_day	hour_of_day	distance_from_home_km	is_fraud
0	522	139.029780	1	18	7.245488	0
1	738	197.581822	4	7	29.843347	0
2	741	20.641438	5	20	7.599835	0
3	661	56.887202	1	19	24.628189	0
4	412	36.608715	3	19	8.770329	0

3. Exploratory Data Analysis

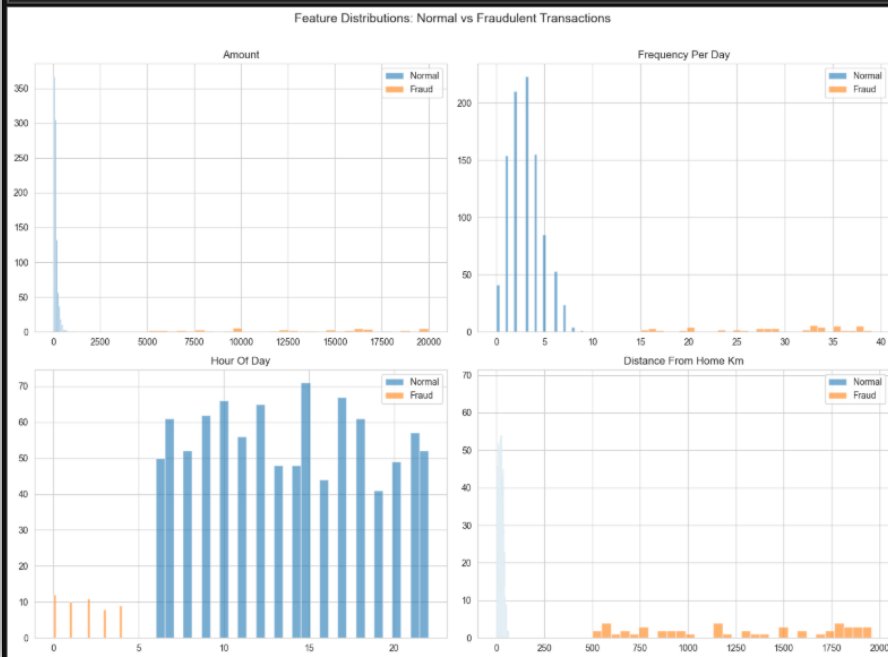
```

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

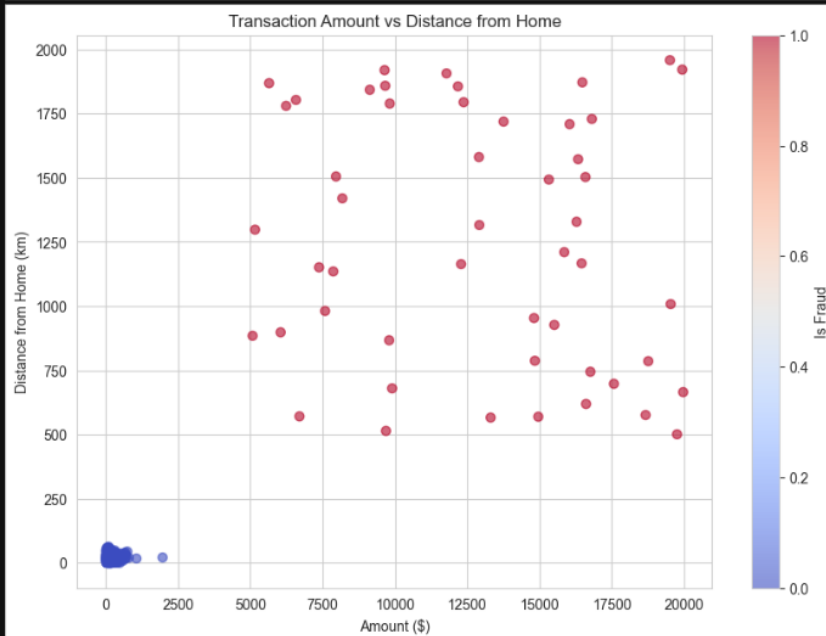
for ax, col in zip(axes.flatten(), ['amount', 'frequency_per_day', 'hour_of_day', 'distance_from_home_km']):
    for label, grp in df.groupby('is_fraud'):
        ax.hist(grp[col], bins=30, alpha=0.6, label=f'Fraud' if label else 'Normal')
    ax.set_title(col.replace('_', ' ').title())
    ax.legend()

plt.suptitle('Feature Distributions: Normal vs Fraudulent Transactions', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()

```



```
plt.figure(figsize=(10, 7))
scatter = plt.scatter(df['amount'], df['distance_from_home_km'],
                     c=df['is_fraud'], cmap='coolwarm', alpha=0.6, s=40)
plt.colorbar(scatter, label='Is Fraud')
plt.title('Transaction Amount vs Distance from Home')
plt.xlabel('Amount ($)')
plt.ylabel('Distance from Home (km)')
plt.show()
```



4. Feature Scaling

```
features = ['amount', 'frequency_per_day', 'hour_of_day', 'distance_from_home_km']
X = df[features]
y_true = df['is_fraud']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print('Scaling done. Shape:', X_scaled.shape)
```

Scaling done. Shape: (1000, 4)

5. Method 1 — Distance-Based (Mahalanobis Distance)

```
# Mahalanobis distance from distribution center
cov_matrix = np.cov(X_scaled.T)
inv_cov = np.linalg.inv(cov_matrix)
mean_vec = np.mean(X_scaled, axis=0)

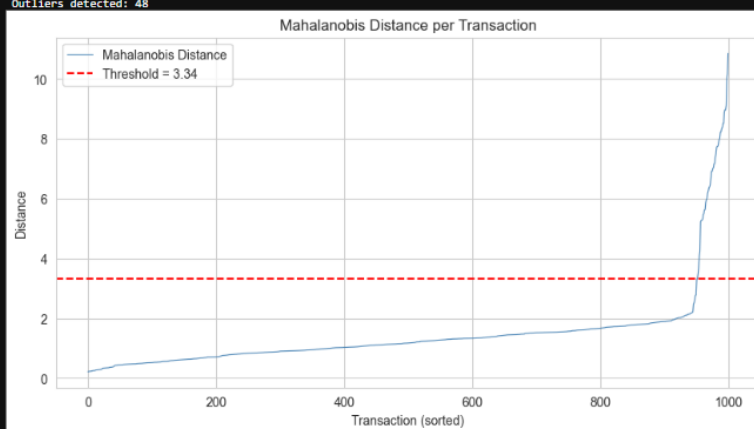
maha_dist = np.array([mahalanobis(row, mean_vec, inv_cov) for row in X_scaled])
df['mahalanobis_dist'] = maha_dist

# Chi-squared threshold (95% confidence, 4 degrees of freedom)
chi2_threshold = stats.chi2.ppf(0.975, df=len(features))
maha_threshold = np.sqrt(chi2_threshold)

df['mahal_outlier'] = (df['mahalanobis_dist'] > maha_threshold).astype(int)
print(f'Mahalanobis threshold: {maha_threshold:.2f}')
print(f'Outliers detected: {df["mahal_outlier"].sum()}')

plt.figure(figsize=(10, 5))
plt.plot(sorted(maha_dist), 'steelblue', linewidth=0.8, label='Mahalanobis Distance')
plt.axhline(y=maha_threshold, color='red', linestyle='--', label=f'Threshold = {maha_threshold:.2f}')
plt.title('Mahalanobis Distance per Transaction')
plt.xlabel('Transaction (sorted)')
plt.ylabel('Distance')
plt.legend()
plt.show()
```

Mahalanobis threshold: 3.34
Outliers detected: 48



6. Method 2 — Local Outlier Factor (LOF)

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
lof_pred = lof.fit_predict(X_scaled)
df['lof_outlier'] = (lof_pred == -1).astype(int)

print(f'LOF Outliers detected: {df["lof_outlier"].sum()}')
print(classification_report(y_true, df['lof_outlier'], target_names=['Normal', 'Fraud']))
```

LOF Outliers detected: 50

	precision	recall	f1-score	support
Normal	0.95	0.95	0.95	950
Fraud	0.04	0.04	0.04	50
accuracy			0.90	1000
macro avg	0.49	0.49	0.49	1000
weighted avg	0.90	0.90	0.90	1000

7. Method 3 — Isolation Forest

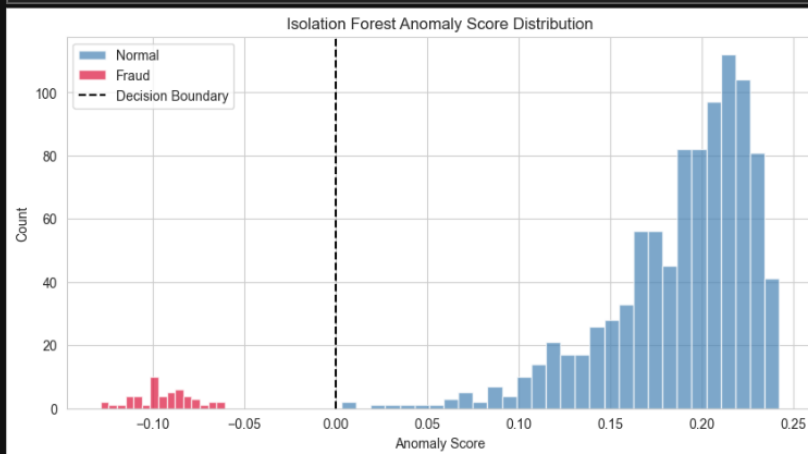
```
iso_forest = IsolationForest(n_estimators=200, contamination=0.05, random_state=42)
iso_pred = iso_forest.fit_predict(X_scaled)
df['iforest_outlier'] = (iso_pred == -1).astype(int)
df['anomaly_score'] = iso_forest.decision_function(X_scaled)

print(f'Isolation Forest Outliers detected: {df["iforest_outlier"].sum()}')
print(classification_report(y_true, df['iforest_outlier'], target_names=['Normal', 'Fraud']))
```

Isolation Forest Outliers detected: 50

	precision	recall	f1-score	support
Normal	1.00	1.00	1.00	950
Fraud	1.00	1.00	1.00	50
accuracy			1.00	1000
macro avg	1.00	1.00	1.00	1000
weighted avg	1.00	1.00	1.00	1000

```
# Isolation Forest - Anomaly Score Distribution
plt.figure(figsize=(10, 5))
normal_scores = df.loc[df['is_fraud']==0, 'anomaly_score']
fraud_scores = df.loc[df['is_fraud']==1, 'anomaly_score']
plt.hist(normal_scores, bins=30, alpha=0.7, color='steelblue', label='Normal')
plt.hist(fraud_scores, bins=15, alpha=0.7, color='crimson', label='Fraud')
plt.axvline(x=0, color='black', linestyle='--', label='Decision Boundary')
plt.title('Isolation Forest Anomaly Score Distribution')
plt.xlabel('Anomaly Score')
plt.ylabel('Count')
plt.legend()
plt.show()
```



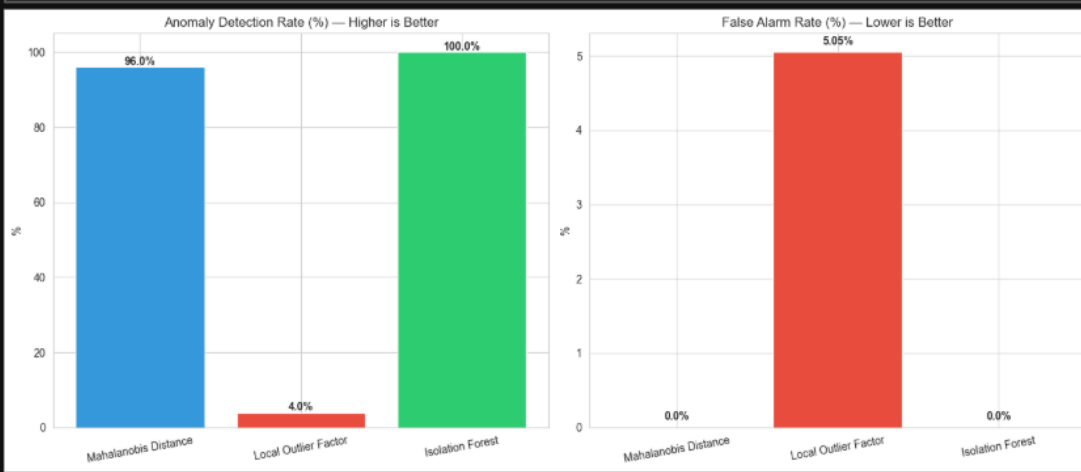
```
# Visual KPI comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

methods = results['Method']
x = np.arange(len(methods))

axes[0].bar(x, results['Anomaly_Detection_Rate'], color=['#3498db', '#e74c3c', '#2ecc71'], edgecolor='white')
axes[0].set_title('Anomaly Detection Rate (%) — Higher is Better')
axes[0].set_xticks(x)
axes[0].set_xticklabels(methods, rotation=10)
axes[0].set_ylabel('%')
for i, v in enumerate(results['Anomaly_Detection_Rate']):
    axes[0].text(i, v+0.5, f'{v}%', ha='center', fontweight='bold')

axes[1].bar(x, results['False_Alarm_Rate'], color=['#3498db', '#e74c3c', '#2ecc71'], edgecolor='white')
axes[1].set_title('False Alarm Rate (%) — Lower is Better')
axes[1].set_xticks(x)
axes[1].set_xticklabels(methods, rotation=10)
axes[1].set_ylabel('%')
for i, v in enumerate(results['False_Alarm_Rate']):
    axes[1].text(i, v+0.1, f'{v}%', ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



Lab 14

Data Scraping & Sentiment Analysis

Objective

To understand the basics of data scraping and sentiment analysis and how they are used in data mining to collect and analyze textual data from online sources.

1. Introduction

Data Scraping and Sentiment Analysis are important techniques in modern data mining. Data scraping is used to collect data from websites or online sources, while sentiment analysis is used to determine the emotional tone behind text data such as positive, negative, or neutral opinions.

These techniques are widely used in social media analysis, market research, and customer feedback analysis.

2. Data Scraping

2.1 Concept

Data scraping is the process of extracting large amounts of data from websites automatically using tools or scripts.

2.2 Sources of Data

- Websites and blogs
- Social media platforms
- Online reviews
- News portals
- E-commerce sites

2.3 Process of Data Scraping

- Identify target website.
- Extract required data (text, images, or tables).
- Store data in structured format (CSV, database, etc.).
- Clean and preprocess collected data.

3. Sentiment Analysis

3.1 Concept

Sentiment analysis is a natural language processing technique used to identify and classify opinions or emotions expressed in text.

3.2 Types of Sentiment

- Positive sentiment
- Negative sentiment
- Neutral sentiment

3.3 Working of Sentiment Analysis

- Collect text data (reviews, tweets, comments).
- Preprocess text (remove noise, stopwords, punctuation).
- Convert text into numerical features.
- Apply machine learning or lexicon-based methods.
- Classify sentiment as positive, negative, or neutral.

1. Install & Import Libraries

```
# Uncomment to install if needed
# !pip install textblob vaderSentiment wordCloud BeautifulSoup4 requests

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import re, string, warnings
warnings.filterwarnings('ignore')

from textblob import TextBlob
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from wordcloud import WordCloud
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from collections import Counter

plt.rcParams['figure.figsize'] = (10, 6)
sns.set_style('whitegrid')
print('All libraries imported!')

All libraries imported!
```

2. Web Scraping Demo (BeautifulSoup) ¶

```
# --- WEB SCRAPING DEMO ---
# In a real scenario, you would scrape live product reviews.
# Here we demonstrate the scraping pipeline using mock HTML.

from bs4 import BeautifulSoup

mock_html = """
<html><body>
<div class="review"><span class="rating">5</span><p class="text">Absolutely amazing product! Exceeded all my expectations. Will buy again!</p></div>
<div class="review"><span class="rating">1</span><p class="text">Terrible quality. Broke after two days. Very disappointed with this purchase.</p>
<div class="review"><span class="rating">4</span><p class="text">Pretty good overall. Delivery was fast and packaging was nice.</p></div>
<div class="review"><span class="rating">2</span><p class="text">Not worth the price. The description was misleading. Customer service was unhelpful.</p>
<div class="review"><span class="rating">5</span><p class="text">Best purchase ever! Highly recommend to everyone looking for great value.</p></div>
</body></html>
"""

soup = BeautifulSoup(mock_html, 'html.parser')
scraped = []
for r in soup.find_all('div', class_='review'):
    scraped.append({
        'rating': int(r.find('span', class_='rating').text),
        'text': r.find('p', class_='text').text.strip()
    })

scraped_df = pd.DataFrame(scraped)
print('Scraped reviews:')
print(scraped_df)
```

```
Scraped reviews:
   rating                                     text
0       5  Absolutely amazing product! Exceeded all my ex...
1       1  Terrible quality. Broke after two days. Very d...
2       4  Pretty good overall. Delivery was fast and pac...
3       2  Not worth the price. The description was misle...
4       5  Best purchase ever! Highly recommend to everyo...
```

3. Load Full Review Dataset

```
# Extended synthetic dataset of 300 product reviews
positive_reviews = [
    "Absolutely love this product! Best purchase I have made this year.",
    "Exceeded all expectations, fast delivery and great quality!",
    "Highly recommend this to everyone. Works perfectly.",
    "Amazing value for money. I'm so happy with this purchase.",
    "Fantastic product! Will definitely buy again and again.",
    "Five stars without hesitation. Superb quality and quick shipping.",
    "Outstanding! Customer service was excellent and product is top notch.",
    "Really impressed with how well this works. Very satisfied customer here.",
    "Brilliant product, sturdy build and looks exactly as described.",
    "Perfect in every way. Could not be happier with this item.",
]

negative_reviews = [
    "Terrible product, broke after one day. Complete waste of money.",
    "Very disappointed. Does not work as advertised. Poor quality.",
    "Worst purchase ever. Returning this immediately. Absolutely awful.",
    "Do not buy this. It is defective and customer service is unhelpful.",
    "Extremely poor quality. Packaging was damaged and item was broken.",
    "Not what I expected at all. Very misleading product description.",
    "Garbage product. Stopped working after two uses. Total disappointment.",
    "Horrible experience from start to finish. Would not recommend to anyone.",
    "Cheap materials and overpriced. Complete scam. Avoid at all costs.",
    "Disgusting quality. Smells bad and looks nothing like the photo.",
]

neutral_reviews = [
    "It is okay. Nothing special but does the job I suppose.",
    "Average product. Not good not bad. Delivery was on time.",
    "It works as described. Nothing more nothing less.",
    "Reasonable quality for the price. Might buy again if on sale.",
    "It is acceptable. I have seen better but also much worse products.",
    "Decent product overall. Some minor issues but manageable.",
    "Works fine. Standard product with no surprises either way.",
    "Neutral on this one. Does what it says on the box, barely.",
    "Not bad not great. Typical product in this price range.",
    "Middle of the road. Would consider alternatives before repurchasing.",
]
```

```

np.random.seed(42)
all_reviews = (positive_reviews * 10) + (negative_reviews * 10) + (neutral_reviews * 10)
all_labels = ([ 'positive' ] * 100) + ([ 'negative' ] * 100) + ([ 'neutral' ] * 100)

# Add noise
shuffle_idx = np.random.permutation(300)
df = pd.DataFrame({
    'review_id': range(1, 301),
    'text': np.array(all_reviews)[shuffle_idx],
    'true_label': np.array(all_labels)[shuffle_idx],
    'star_rating': np.where(np.array(all_labels)[shuffle_idx]=='positive',
                           np.random.randint(4,6,300),
                           np.where(np.array(all_labels)[shuffle_idx]=='negative',
                                   np.random.randint(1,3,300), 3))
})

print('Dataset shape:', df.shape)
print('Label distribution:')
print(df['true_label'].value_counts())
df.head()

```

Dataset shape: (300, 4)

Label distribution:

true_label

neutral 100

negative 100

positive 100

Name: count, dtype: int64

	review_id	text	true_label	star_rating
0	1	Reasonable quality for the price. Might buy ag...	neutral	3
1	2	Works fine. Standard product with no surprises...	neutral	3
2	3	Worst purchase ever. Returning this immediatel...	negative	2
3	4	Perfect in every way. Could not be happier wit...	positive	4
4	5	Reasonable quality for the price. Might buy ag...	neutral	3

4. Text Cleaning & Preprocessing

```

def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text) # remove URLs
    text = re.sub(r'@\w+|#\w+', '', text) # remove mentions/hashtags
    text = re.sub(r'[^a-z\s]', '', text) # keep only letters
    text = re.sub(r'\s+', ' ', text).strip() # remove extra spaces
    return text

df['clean_text'] = df['text'].apply(clean_text)

print('Sample cleaning:')
for orig, clean in zip(df['text'][:3], df['clean_text'][:3]):
    print(f' Original : {orig}')
    print(f' Cleaned  : {clean}')
    print()

```

Sample cleaning:

Original : Reasonable quality for the price. Might buy again if on sale.
Cleaned : reasonable quality for the price might buy again if on sale

Original : Works fine. Standard product with no surprises either way.
Cleaned : works fine standard product with no surprises either way

Original : Worst purchase ever. Returning this immediately. Absolutely awful.
Cleaned : worst purchase ever returning this immediately absolutely awful

6. Sentiment Analysis — TextBlob

```
def textblob_sentiment(text):
    blob = TextBlob(text)
    polarity = blob.sentiment.polarity
    if polarity > 0.05: return 'positive', polarity
    elif polarity < -0.05: return 'negative', polarity
    else: return 'neutral', polarity

tb_results = df['clean_text'].apply(textblob_sentiment)
df['tb_sentiment'] = tb_results.apply(lambda x: x[0])
df['tb_polarity'] = tb_results.apply(lambda x: x[1])

print('TextBlob Sentiment Distribution:')
print(df['tb_sentiment'].value_counts())
```

```
TextBlob Sentiment Distribution:
tb_sentiment
positive    150
negative     90
neutral     60
Name: count, dtype: int64
```

7. Sentiment Analysis — VADER

```
analyzer = SentimentIntensityAnalyzer()

def vader_sentiment(text):
    scores = analyzer.polarity_scores(text)
    compound = scores['compound']
    if compound >= 0.05: return 'positive', compound
    elif compound <= -0.05: return 'negative', compound
    else: return 'neutral', compound

vader_results = df['text'].apply(vader_sentiment)
df['vader_sentiment'] = vader_results.apply(lambda x: x[0])
df['vader_compound'] = vader_results.apply(lambda x: x[1])

print('VADER Sentiment Distribution:')
print(df['vader_sentiment'].value_counts())
```

```
VADER Sentiment Distribution:
vader_sentiment
negative    130
positive    100
neutral     70
Name: count, dtype: int64
```

8. ML Classifier — TF-IDF + Logistic Regression

```
from sklearn.preprocessing import LabelEncoder

X_text = df['clean_text']
y = df['true_label']

X_train, X_test, y_train, y_test = train_test_split(X_text, y, test_size=0.25, random_state=42, stratify=y)

tfidf = TfidfVectorizer(max_features=500, ngram_range=(1,2), stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

clf = LogisticRegression(max_iter=1000, random_state=42, C=1.0)
clf.fit(X_train_tfidf, y_train)
y_pred = clf.predict(X_test_tfidf)

print('TF-IDF + Logistic Regression Report:')
print(classification_report(y_test, y_pred))
```

```
TF-IDF + Logistic Regression Report:
              precision    recall  f1-score   support

negative       1.00        1.00        1.00         25
neutral        1.00        1.00        1.00         25
positive       1.00        1.00        1.00         25

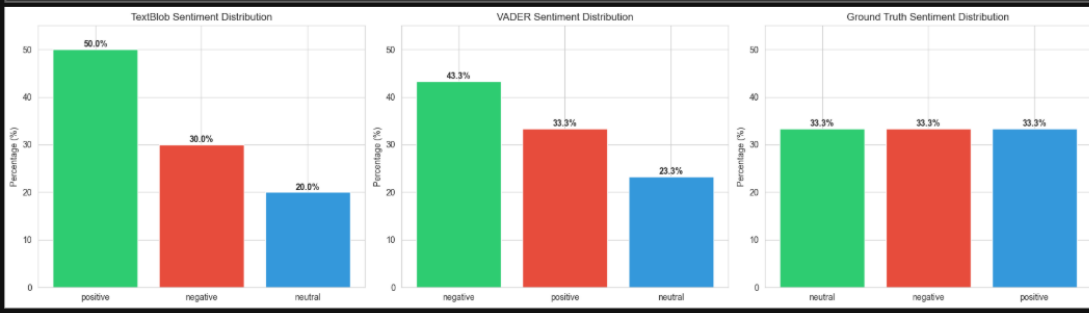
accuracy              1.00         75
macro avg           1.00        1.00        1.00         75
weighted avg        1.00        1.00        1.00         75
```

9. KPI — Sentiment % Distribution & Polarity Score

```
# KPI 1: Sentiment % Distribution
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, col, title in zip(axes, ['tb_sentiment', 'vader_sentiment', 'true_label'],
                           ['TextBlob', 'VADER', 'Ground Truth']):
    counts = df[col].value_counts()
    pct = counts / counts.sum() * 100
    colors = ['#2ecc71', '#e74c3c', '#3498db']
    bars = ax.bar(pct.index, pct.values, color=colors[:len(pct)], edgecolor='white')
    for bar, pct in zip(bars, pct.values):
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
                f'{pct:.1f}%', ha='center', fontweight='bold')
    ax.set_title(f'{title} Sentiment Distribution')
    ax.set_ylabel('Percentage (%)')
    ax.set_ylim(0, 55)

plt.tight_layout()
plt.show()
```

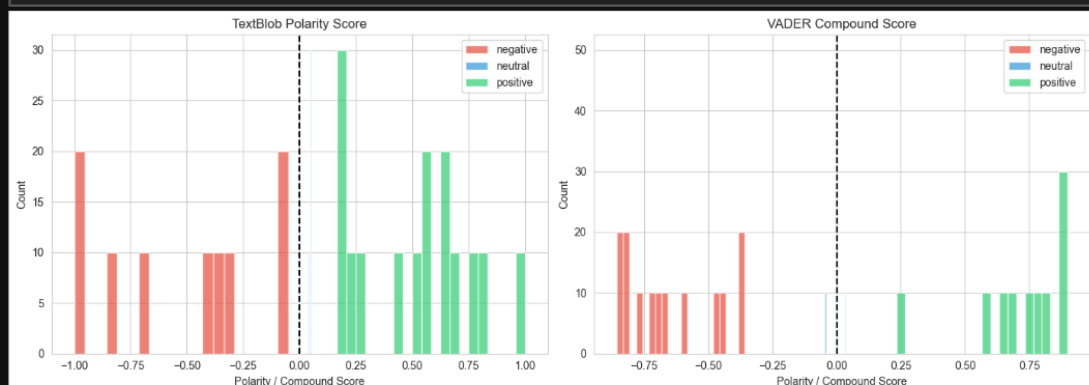


```
# KPI 2: Polarity Score Distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
pal = {'positive': '#2ecc71', 'negative': '#e74c3c', 'neutral': '#3498db'}

for ax, col, score_col, title in [
    (axes[0], 'tb_sentiment', 'tb_polarity', 'TextBlob Polarity Score'),
    (axes[1], 'vader_sentiment', 'vader_compound', 'VADER Compound Score')
]:
    for sentiment, grp in df.groupby(col):
        ax.hist(grp[score_col], bins=20, alpha=0.7, color=pal[sentiment], label=sentiment)
    ax.axvline(x=0, color='black', linestyle='--', linewidth=1.5)
    ax.set_title(title)
    ax.set_xlabel('Polarity / Compound Score')
    ax.set_ylabel('Count')
    ax.legend()

plt.tight_layout()
plt.show()

print('\n\nMean Polarity Scores by True Sentiment:')
print(df.groupby('true_label')[['tb_polarity', 'vader_compound']].mean().round(4))
```



Lab 15

Performance Comparison of Models

Objective

To understand how to evaluate and compare different machine learning models using performance metrics in data mining.

1. Introduction

In data mining and machine learning, multiple models are often trained to solve the same problem. However, not all models perform equally well. Therefore, it is important to evaluate and compare models using standard performance metrics to select the best one.

Model comparison helps in identifying which algorithm gives the highest accuracy, better predictions, and more reliable results on unseen data.

2. Importance of Model Comparison

- Helps select the best performing model.
- Improves prediction accuracy.
- Ensures better decision-making.
- Reduces overfitting or underfitting issues.
- Increases reliability of results.

3. Performance Metrics

3.1 Accuracy

- Measures the percentage of correct predictions.
- Higher accuracy indicates better performance.

3.2 Precision

- Measures how many predicted positives are actually correct.
- Important in classification problems.

3.3 Recall

- Measures how many actual positives are correctly predicted.
- Useful when missing positive cases is critical.

3.4 F1 Score

- Harmonic mean of precision and recall.
- Useful when data is imbalanced.

3.5 Confusion Matrix

- Table showing correct and incorrect predictions.
- Includes True Positive, True Negative, False Positive, False Negative.

4. Model Comparison Process

Step 1: Train Models

- Train multiple machine learning models on the same dataset.

Step 2: Predict Results

- Generate predictions using each model.

Step 3: Evaluate Metrics

- Calculate accuracy, precision, recall, and F1-score.

Step 4: Compare Results

- Compare all models based on evaluation metrics.

Step 5: Select Best Model

- Choose the model with the best overall performance.

5. Common Models for Comparison

- Decision Tree
- Naïve Bayes
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Random Forest

Step 1 — Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Preprocessing
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline

# 8 ML Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    AdaBoostClassifier, BaggingClassifier
)
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)

print('✅ All libraries imported successfully!')
✅ All libraries imported successfully!
```

Step 2 — Dataset: E-Commerce Customer Churn

```
np.random.seed(42)
n = 2000

data = pd.DataFrame({
    'CustomerID' : range(1, n+1),
    'Age' : np.random.randint(18, 70, n),
    'Gender' : np.random.choice(['Male', 'Female'], n),
    'Tenure_Months' : np.random.randint(1, 60, n),
    'Monthly_Spend' : np.round(np.random.uniform(20, 500, n), 2),
    'Num_Orders' : np.random.randint(1, 100, n),
    'Return_Rate' : np.round(np.random.uniform(0, 0.5, n), 2),
    'Support_Tickets' : np.random.randint(0, 10, n),
    'Last_Purchase_Days' : np.random.randint(1, 365, n),
    'Preferred_Category' : np.random.choice(['Electronics', 'Clothing', 'Groceries', 'Books'], n),
    'Discount_Used' : np.random.choice([0, 1], n),
    'Loyalty_Points' : np.random.randint(0, 5000, n),
})

# Inject realistic churn logic
churn_prob = (
    0.3
    + 0.3 * (data['Last_Purchase_Days'] > 180).astype(int)
    + 0.2 * (data['Support_Tickets'] > 5).astype(int)
    - 0.2 * (data['Tenure_Months'] > 24).astype(int)
    - 0.1 * (data['Loyalty_Points'] > 2500).astype(int)
    + 0.1 * (data['Return_Rate'] > 0.3).astype(int)
).clip(0.05, 0.95)

data['Churn'] = np.random.binomial(1, churn_prob)

# Introduce missing values (realistic)
for col in ['Monthly_Spend', 'Return_Rate', 'Loyalty_Points']:
    mask = np.random.choice([True, False], n, p=[0.04, 0.96])
    data.loc[mask, col] = np.nan

print(f'Dataset shape: {data.shape}')
print(f'Churn rate : {data["Churn"].mean()*100:.1f}%')
data.head()
```

```
Dataset shape: (2000, 13)
Churn rate : 39.8%
```

	CustomerID	Age	Gender	Tenure_Months	Monthly_Spend	Num_Orders	Return_Rate	Support_Tickets	Last_Purchase_Days	Preferred_Category	Discount_Used
0	1	56	Male	21	395.34	3	0.41	9	283	Groceries	0
1	2	69	Female	5	380.84	42	0.27	5	106	Clothing	1
2	3	46	Female	48	405.44	21	0.46	0	300	Groceries	1
3	4	32	Male	20	268.64	99	0.47	5	107	Clothing	1
4	5	60	Female	37	87.36	40	0.03	2	36	Electronics	1

Step 3 — Exploratory Data Analysis (EDA)

```
print('=== Dataset Info ===')
data.info()
print('\n=== Missing Values ===')
print(data.isnull().sum())
print('\n=== Class Distribution ===')
print(data['Churn'].value_counts())
```

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   CustomerID            2000 non-null   int64  
1   Age                   2000 non-null   int64  
2   Gender                2000 non-null   object  
3   Tenure_Months         2000 non-null   int64  
4   Monthly_Spend         1921 non-null   float64  
5   Num_Orders            2000 non-null   int64  
6   Return_Rate           1921 non-null   float64  
7   Support_Tickets        2000 non-null   int64  
8   Last_Purchase_Days    2000 non-null   int64  
9   Preferred_Category    2000 non-null   object  
10  Discount_Used         2000 non-null   int64  
11  Loyalty_Points        1927 non-null   float64  
12  Churn                 2000 non-null   int64  
dtypes: float64(3), int64(8), object(2)
memory usage: 283.3+ KB
```

```
=== Missing Values ===
CustomerID    0
Age           0
Gender        0
Tenure_Months 0
Monthly_Spend 79
Num_Orders    0
Return_Rate   79
Support_Tickets 0
Last_Purchase_Days 0
Preferred_Category 0
Discount_Used 0
Loyalty_Points 73
Churn         0
dtype: int64
```

```
=== Class Distribution ===
Churn
0    1285
1     795
Name: count, dtype: int64
```

```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle('EDA - E-Commerce Customer Churn Dataset', fontsize=15, fontweight='bold')

# 1. Churn distribution
churn_counts = data['Churn'].value_counts()
axes[0,0].bar(['Retained (0)', 'Churned (1)'], churn_counts.values,
              color=['#2ecc71', '#e74c3c'], edgecolor='white', linewidth=1.5)
axes[0,0].set_title('Churn Distribution')
axes[0,0].set_ylabel('Count')
for i, v in enumerate(churn_counts.values):
    axes[0,0].text(i, v + 10, str(v), ha='center', fontweight='bold')

# 2. Age distribution by churn
data[data['Churn']==0]['Age'].hist(ax=axes[0,1], alpha=0.6, color='#2ecc71', label='Retained', bins=20)
data[data['Churn']==1]['Age'].hist(ax=axes[0,1], alpha=0.6, color='#e74c3c', label='Churned', bins=20)
axes[0,1].set_title('Age Distribution by Churn')
axes[0,1].legend()

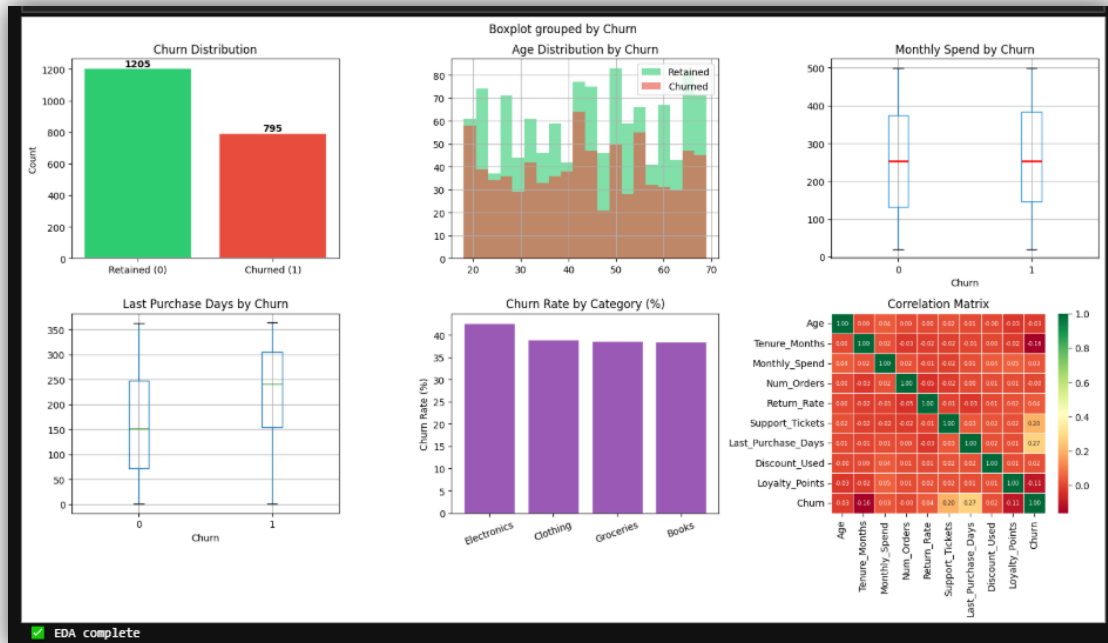
# 3. Monthly Spend by Churn
data.boxplot(column='Monthly_Spend', by='Churn', ax=axes[0,2],
              boxprops=dict(color='#3498db'), medianprops=dict(color='red', linewidth=2))
axes[0,2].set_title('Monthly Spend by Churn')
plt.sca(axes[0,2])
plt.title('Monthly Spend by Churn')

# 4. Last Purchase Days vs Churn
data.boxplot(column='Last_Purchase_Days', by='Churn', ax=axes[1,0])
axes[1,0].set_title('Last Purchase Days by Churn')
plt.sca(axes[1,0])
plt.title('Last Purchase Days by Churn')

# 5. Preferred Category churn rate
cat_churn = data.groupby('Preferred_Category')['Churn'].mean().sort_values(ascending=False)
axes[1,1].bar(cat_churn.index, cat_churn.values * 100, color='#9b59b6', edgecolor='white')
axes[1,1].set_title('Churn Rate by Category (%)')
axes[1,1].set_ylabel('Churn Rate (%)')
axes[1,1].tick_params(axis='x', rotation=20)

# 6. Correlation heatmap (numeric)
numeric_cols = data.select_dtypes(include=np.number).drop(columns=['CustomerID']).columns
corr = data[numeric_cols].corr()
sns.heatmap(corr, ax=axes[1,2], annot=True, fmt='.2f', cmap='RdYlGn',
            linewidths=0.5, annot_kws={'size': 6})
axes[1,2].set_title('Correlation Matrix')

plt.tight_layout()
plt.savefig('tmp/eda.png', dpi=120, bbox_inches='tight')
plt.show()
print('🟢 EDA complete')
```



Step 4 — Data Preprocessing

```
df = data.drop(columns=['CustomerID']).copy()

# 4a. Handle missing values
num_cols = df.select_dtypes(include=np.number).columns.tolist()
num_cols.remove('Churn')
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# 4b. Encode categorical
le = LabelEncoder()
for col in ['Gender', 'Preferred_Category']:
    df[col] = le.fit_transform(df[col])

# 4c. Feature / Target split
X = df.drop('Churn', axis=1)
y = df['Churn']

# 4d. Train / Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 4e. Feature scaling
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f'Train size : {X_train.shape[0]} samples')
print(f'Test size : {X_test.shape[0]} samples')
print(f'Features : {X.shape[1]}')
print('✅ Preprocessing done - no missing values:', df.isnull().sum().sum())

Train size : 1600 samples
Test size : 400 samples
Features : 11
✅ Preprocessing done - no missing values: 0
```

Step 5 — Train 8 Machine Learning Models

```
models = {
    '1. Logistic Regression' : LogisticRegression(max_iter=1000, random_state=42),
    '2. Decision Tree' : DecisionTreeClassifier(max_depth=6, random_state=42),
    '3. Random Forest' : RandomForestClassifier(n_estimators=100, random_state=42),
    '4. Gradient Boosting' : GradientBoostingClassifier(n_estimators=100, random_state=42),
    '5. AdaBoost' : AdaBoostClassifier(n_estimators=100, random_state=42),
    '6. Bagging Classifier' : BaggingClassifier(n_estimators=100, random_state=42),
    '7. Support Vector Machine' : SVC(kernel='rbf', probability=True, random_state=42),
    '8. K-Nearest Neighbors' : KNeighborsClassifier(n_neighbors=7),
}

# Models that need scaled data
scale_needed = {'1. Logistic Regression', '7. Support Vector Machine', '8. K-Nearest Neighbors'}

results = []
trained_models = {}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print(f'{"Model":<30} {"Acc":>7} {"Prec":>7} {"Rec":>7} {"F1":>7} {"AUC":>7} {"CV-Acc":>8}')
print('-' * 80)

for name, model in models.items():
    Xtr = X_train_s if name in scale_needed else X_train
    Xte = X_test_s if name in scale_needed else X_test

    model.fit(Xtr, y_train)
    y_pred = model.predict(Xte)
    y_prob = model.predict_proba(Xte)[:,1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)
    cv_sc = cross_val_score(model, Xtr, y_train, cv=cv, scoring='accuracy').mean()

    results.append({'Model': name, 'Accuracy': acc, 'Precision': prec,
                  'Recall': rec, 'F1-Score': f1, 'ROC-AUC': auc, 'CV Accuracy': cv_sc})
    trained_models[name] = (model, Xte, y_pred, y_prob)

print(f'{"name":<30} {"acc":>7.4f} {"prec":>7.4f} {"rec":>7.4f} {"f1":>7.4f} {"auc":>7.4f} {"cv_sc":>8.4f}')

print('\n✅ All 8 models trained!')
```

Model	Acc	Prec	Rec	F1	AUC	CV-Acc
1. Logistic Regression	0.6775	0.6250	0.4717	0.5376	0.7248	0.6887
2. Decision Tree	0.6650	0.5862	0.5346	0.5592	0.6843	0.6400
3. Random Forest	0.6525	0.5794	0.4591	0.5123	0.7118	0.6894
4. Gradient Boosting	0.6525	0.5735	0.4906	0.5288	0.7240	0.6756
5. AdaBoost	0.6725	0.6111	0.4843	0.5484	0.7505	0.7006
6. Bagging Classifier	0.6650	0.5850	0.5409	0.5621	0.7123	0.6700
7. Support Vector Machine	0.6600	0.6036	0.4214	0.4963	0.6977	0.6000
8. K-Nearest Neighbors	0.6475	0.5703	0.4591	0.5087	0.6385	0.6412

✅ All 8 models trained!

```
# Visual comparison bar chart
res_plot = pd.DataFrame(results).sort_values('Accuracy', ascending=False)
short_names = [n.split('.')[1] for n in res_plot['Model']]

metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']
colors = ['#3498db', '#2ecc71', '#e74c3c', '#f39c12', '#9b59b6']

x = np.arange(len(short_names))
width = 0.15

fig, ax = plt.subplots(figsize=(16, 6))
for i, (metric, color) in enumerate(zip(metrics, colors)):
    offset = (i - 2) * width
    bars = ax.bar(x + offset, res_plot[metric].values, width, label=metric,
                  color=color, alpha=0.85, edgecolor='white')

ax.set_xlabel('Model', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('📊 Model Comparison - All Metrics', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(short_names, rotation=20, ha='right', fontsize=10)
ax.set_ylim(0, 1.05)
ax.legend(loc='lower right')
ax.axhline(0.8, color='gray', linestyle='--', linewidth=0.8, alpha=0.5)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('/tmp/model_comparison.png', dpi=120, bbox_inches='tight')
plt.show()
```



Step 7 — KPI: Best Accuracy & Deep Dive

```
res_num = pd.DataFrame(results)
best_row = res_num.loc[res_num['Accuracy'].idxmax()]
best_name = best_row['Model']
best_acc = best_row['Accuracy']

print('📌 BEST MODEL')
print('=' * 50)
print(f' Model           : {best_name}')
print(f' Accuracy          : {best_acc:.4f} ({best_acc*100:.2f}%)')
print(f' Precision         : {best_row["Precision"]:.4f}')
print(f' Recall            : {best_row["Recall"]:.4f}')
print(f' F1-Score          : {best_row["F1-Score"]:.4f}')
print(f' ROC-AUC           : {best_row["ROC-AUC"]:.4f}')
print(f' CV Accuracy       : {best_row["CV Accuracy"]:.4f}')
print('=' * 50)
```

```
📌 BEST MODEL
=====
Model           : 1. Logistic Regression
Accuracy        : 0.6775 (67.75%)
Precision       : 0.6250
Recall          : 0.4717
F1-Score        : 0.5376
ROC-AUC         : 0.7248
CV Accuracy     : 0.6887
=====
```

Step 8 — KPI: Business Recommendation

```
# Business Impact Calculation
total_customers = 50_000      # hypothetical customer base
avg_clv         = 1_200       # $ customer lifetime value
retention_cost  = 50           # $ per targeted intervention
retention_success = 0.40      # % of flagged churners saved

churn_rate      = data['Churn'].mean()
expected_churners = int(total_customers * churn_rate)
model_recall     = best_row['Recall']
model_precision  = best_row['Precision']

true_churners_caught = int(expected_churners * model_recall)
false_alarms         = int(true_churners_caught / model_precision * (1 - model_precision))
total_contacted      = true_churners_caught + false_alarms
customers_saved       = int(true_churners_caught * retention_success)
revenue_saved         = customers_saved * avg_clv
campaign_cost         = total_contacted * retention_cost
net_benefit           = revenue_saved - campaign_cost
roi                   = (net_benefit / campaign_cost) * 100

print('=' * 60)
print('📌 BUSINESS IMPACT ANALYSIS')
print('=' * 60)
print(f' Total Customer Base           : {total_customers:,}')
print(f' Estimated Annual Churners     : {expected_churners:,} ({churn_rate*100:.1f}%)')
print(f' Model Recall (Sensitivity)    : {model_recall*100:.1f}%)')
print(f' Churners Correctly Flagged    : {true_churners_caught:,}')
print(f' False Alarms (Non-churners)   : {false_alarms:,}')
print(f' Total Customers Contacted     : {total_contacted:,}')
print(f' Customers Retained (40%)      : {customers_saved:,}')
print(f' Revenue Saved                 : ${revenue_saved:,.0f}')
print(f' Campaign Cost                 : ${campaign_cost:,.0f}')
print(f' Net Benefit                   : ${net_benefit:,.0f}')
print(f' Return on Investment (ROI)    : {roi:.1f}%)')
print('=' * 60)
```

```
📌 BUSINESS IMPACT ANALYSIS
=====
Total Customer Base           : 50,000
Estimated Annual Churners     : 19,875 (39.8%)
Model Recall (Sensitivity)    : 47.2%
Churners Correctly Flagged    : 9,375
False Alarms (Non-churners)   : 5,625
Total Customers Contacted     : 15,000
Customers Retained (40%)      : 3,750
Revenue Saved                 : $4,500,000
Campaign Cost                 : $750,000
Net Benefit                   : $3,750,000
Return on Investment (ROI)    : 500.0%
=====
```